Laying Zombie Vectors to Rest: Per-View Erasure in Graph Indexes

Yizheng Zhao
State Key Laboratory of Novel Software Technology
Nanjing University, China
zhaoyz@nju.edu.cn

Wanjing Ren
State Key Laboratory of Novel Software Technology
Nanjing University, China
renwj@smail.nju.edu.cn

ABSTRACT

Revoking a user's access to a vector should make it vanish from that user's results. In a shared graph index, the standard for high-recall approximate nearest neighbor (ANN) search, it does not. The vector cannot simply be deleted: other users still need it, and true deletion demands costly neighborhood repair, so systems merely flag it. It stays in the graph, steering the revoked user's search, and remains byte-recoverable from memory and snapshots. We call these *zombie vectors*. Every existing remedy is global: leaving the vector lets it keep routing, removing or shredding it erases it for every user at once, and rebuilding per revocation is prohibitively expensive. Erasure is therefore not a structural delete but a per-view property of traversal: after revocation the index must behave, in both results and routing, as if the vector had never entered the revoked user's view, while staying unchanged for everyone else. This guarantee, *per-view erasure*, spans six independent axes (Results, Routing, Physical, Immediate, Verifiable, View-scope); Tombstone, Physical-delete, Periodic-rebuild, and cryptographic shredding each violate at least two. We present Lethe, an index and query-processing design that satisfies all six: revocation-aware traversal with a bypass overlay, shared across principals with the same view, that keeps a revoked vector's authorized neighbors reachable; cryptographic shredding for global physical erasure; and an append-only erasure log. Across clinical, legal, web, and standard ANN benchmarks, Lethe drives per-view leakage to near zero, holds recall near an oracle per-view rebuild, and erases immediately at far lower memory than per-user indexes, scaling to a billion vectors. Erasure belongs where queries are answered, not where data is stored.

PVLDB Reference Format:
Yizheng Zhao and Wanjing Ren. Zombie Vectors. PVLDB, 20(1): XXX-XXX, 2027.doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://anonymous.4open.science/r/lethe/.

1 INTRODUCTION

A single vector index increasingly answers similarity-search queries for many users at once, with access governed by per-user or per-role permissions [29, 57]. A hospital assistant retrieves over a shared embedding of patient notes, but a clinician reassigned off a case must lose access to those notes; an enterprise copilot retrieves over one corporate index, but a reclassified document must disappear for users below its new clearance; a data subject invokes the right to be forgotten [15], and their vector must stop reaching anyone. In each case, access to a vector is *revoked* for some users while the vector itself must remain for others. The system cannot simply delete it.

Graph-based indexes such as HNSW [34] and DiskANN [47] are the standard for high-recall similarity search and the backbone of production vector databases [25, 48]. They are also where revocation goes wrong. Deleting a node from a navigable graph is expensive: its in-neighbors must be re-patched so that traversal still reaches the rest of the graph, at a cost that grows with the node's in-degree [44]. To avoid paying this cost on every deletion, production systems *tombstone*: they mark a vector deleted, filter it from results, and defer the neighborhood repair to a periodic consolidation pass [44]. A tombstoned vector therefore (i) stays physically present and byte-recoverable, in memory and in serialized snapshots, and (ii) keeps serving as a routing waypoint that steers traversal, including the traversal that answers a revoked user's query. We call a revoked vector that still routes a *zombie vector*: dead to the access policy, yet still choosing the living results. The effect is not marginal: on a clustered index, revoking by semantic cluster can shift more than one in six of a user's authorized top-10 answers away from an oracle per-view rebuild (§3).

Why has this gone unaddressed? Work on forgetting data splits along two layers that both skip the index. Secure deletion erases bytes from the storage medium [41] but not the derived structural copies an index keeps; machine unlearning removes a datum's influence from a trained model [3, 6, 20], but a vector index has no model to retrain. Between them sits the index structure itself, and it is empty. Vector-index research treats deletion as a freshness problem, removing vectors globally to protect recall [36, 50–53, 55] or characterizing deletion accuracy without security or per-user scope [54]; cryptographic shredding discards a vector's key [59] but globally, leaving the node routing; and a relational notion of erasure under dependencies [8] does not model graph traversal. No existing method erases a vector from one view while leaving it intact for the rest.

Our starting point is a reframing: erasing a vector from a graph index is not removing its bytes, it is removing its influence on traversal, and that influence is exactly what Tombstone and cryptographic shredding leave behind. In a shared index this influence is per-view, and removing it is what we call *per-view erasure*: after revocation the index must behave, in both its results and routing, as if the vector had never entered the revoked user's view, while remaining unchanged for every other view.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

	Results	Routing	Physical	Immediate	Verifiable	View-scope
Tombstone	✓	✗	✗	✓	✗	✗
Physical-delete	✓	✓	✓	✓	✗	✗
Periodic-rebuild	✓	✓	✗	✗	✗	✗
Cryptographic shredding	✗	✗	✓	✓	✓	✗
Lethe (ours)	✓	✓	✓	✓	✓	✓

Figure 1: Erasure capability of index-maintenance methods on the six axes defined in §4: whether revoked data is removed from query *results*, removed from *routing* (traversal), made *physically* unrecoverable, applied *immediately*, *verifiable*, and scoped to a single *view*. Each prior method fails at least two axes, and only Lethe achieves per-view erasure.

Against this definition we characterize prior methods on six axes (*Results*, *Routing*, *Physical*, *Immediate*, *Verifiable*, *View-scope*), defined in §4: Tombstone, Physical-delete, Periodic-rebuild, and cryptographic shredding each fail at least two, and none achieves per-view scope (Fig. 1).

We then present Lethe, an index and query-processing design that satisfies all six axes through four mechanisms, each tied to the axis it secures. *Revocation-aware traversal* makes "not routable for the revoked view" an invariant of search, so a revoked vector stops steering that view's queries the moment the revocation is recorded (routing, immediacy, view scope). An *effective-view-shared bypass overlay* restores the navigability a revoked vector used to provide, sharing bypass edges across views so recall holds without an index per user (results, navigability, memory). Vector-granular *cryptographic shredding* [59] supplies the physical, global erasure the right to be forgotten requires, and an append-only *erasure log* renders every revocation auditable (physical, verifiable).

We evaluate Lethe on standard ANN benchmarks and access-controlled clinical, legal, and web corpora, against the most recent streaming-index and access-controlled baselines. It drives per-view leakage to near zero, makes globally erased vectors unrecoverable, holds recall close to an oracle per-view rebuild, sustains high revocation throughput, and uses far less memory than per-user indexes at billion-vector scale. This paper makes four **contributions**:

(1) We identify and measure the *zombie vector* phenomenon: in graph vector indexes, revoked data stays physically recoverable and keeps routing live queries (§2, §3).
(2) We formalize *per-view erasure* through six independent axes and prove no standard primitive achieves it, each violating at least two (§4).
(3) We design Lethe, the first to satisfy all six axes, with revocation-aware traversal, an effective-view-shared bypass overlay, vector-granular cryptographic shredding, and an append-only erasure log (§5, §6).
(4) We evaluate Lethe against recent streaming-index and access-controlled baselines, with statistical significance and billion-scale experiments; proofs & full results are in an extended version (§7).

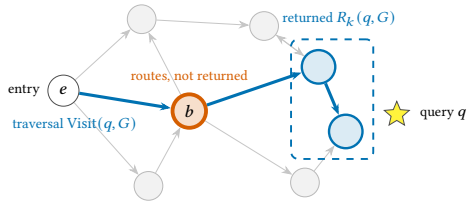

Figure 2: Running example. Greedy search for q visits the nodes on the bold path, $\mathrm{Visit}(q, G)$, but returns only the k nearest, $R_k(q, G)$ (boxed). The bridge b *routes* the query yet is never *returned*. This separation is what makes revocation subtle: once b is revoked, Tombstone filters it from R_k but leaves it routing the revoked user's search (the *zombie* channel of §3), whereas Lethe routes around it. We reuse this index throughout the paper.

2 BACKGROUND AND THREAT MODEL

2.1 Graph-based vector indexes

A graph-based vector index over a dataset $P = \{p_1, \ldots, p_N\}$ of N vectors in d dimensions ($P \subset \mathbb{R}^d$) is a directed graph $G = (P, E)$ in which every node is a vector and each node p stores a short adjacency list of its out-neighbors $N_{\mathrm{out}}(p)$, capped at an out-degree bound M [34, 47]. A query q is answered by greedy best-first traversal: starting from a fixed entry point, the search keeps a frontier of the ef closest candidates seen so far (ef trades accuracy for speed), repeatedly expands the closest unexpanded candidate by visiting its out-neighbors, and stops when the frontier no longer improves; the k closest visited nodes are returned [34]. We write $\mathrm{Visit}(q, G)$ for the set of nodes expanded during this traversal, $R_k(q, G)$ for the k neighbors returned, and measure quality by $\mathrm{Recall@}k = |R_k(q, G) \cap \mathrm{NN}_k(q)|/k$, the fraction of the true k nearest neighbors $\mathrm{NN}_k(q)$ recovered. The index works because G is *navigable*: from the entry point a greedy walk reaches the neighborhood of q in a few hops, which requires that the nodes along the way keep pointing toward the rest of the graph.

A node p is a *routing waypoint* for q when $p \in \mathrm{Visit}(q, G)$: the search passes through it, steered onward by its out-edges. This separation between routing and being returned is central to §4.

2.2 Deletion and the lazy-repair regime

Removing a vector p from G is not free. If p is simply dropped, every in-neighbor $p' \in N_{\mathrm{in}}(p)$ loses an out-edge, and the walks that used to pass through p may no longer reach their target, degrading recall. Restoring navigability requires re-pointing each in-neighbor to suitable survivors, an operation whose cost grows with $|N_{\mathrm{in}}(p)|$ [44]. Because in-degrees are large in a well-connected index, paying this on every deletion is prohibitive, so production systems delete *lazily*: a deleted vector is placed in a tombstone set, excluded from returned results, and its neighborhood repair is batched into a periodic *consolidation* pass that runs only after deletions accumulate [44]. Between deletion and consolidation the tombstoned vector remains a node of G with its edges intact: it is still a routing waypoint, its vector still occupies memory, and it is written into every serialized snapshot. This regime is standard

and, for freshness, effective. §3 shows why it becomes unsafe once deletion means revocation.

2.3 Multi-tenancy and access control

A shared index serves a set of principals U (users or roles). An access policy assigns each vector p an authorized set $A(p) \subseteq U$; principal u's *view* is the sub-dataset $P_u = \{p \in P : u \in A(p)\}$ it is permitted to retrieve. Permissions are organized as a lattice L of roles with partial order $\sqsubseteq$, where $\ell' \sqsubseteq \ell$ means role ℓ subsumes the access of role ℓ'; we then say ℓ *dominates* ℓ'. Each principal u holds a role $\lambda(u) \in L$ and each vector p is tagged with the least role $\lambda(p) \in L$ permitted to see it, so u may retrieve p exactly when $\lambda(p) \sqsubseteq \lambda(u)$; equivalently $A(p) = \{u \in U : \lambda(p) \sqsubseteq \lambda(u)\}$. For any role $\ell \in L$ we write $P_\ell = \{p \in P : \lambda(p) \sqsubseteq \ell\}$ for the vectors *visible at* ℓ; a principal's view is the view of its role, $P_u = P_{\lambda(u)}$, so principals that share a role share a view [29, 57]. Access-controlled vector systems enforce P_u either by partitioning the index per role or tenant [29, 57] or by filtering search with a predicate [5, 21, 39], and relational systems apply row-level security by query rewriting [42]. A *revocation* revoke(u, p) removes u from $A(p)$, shrinking P_u by one vector while leaving $P_{u'}$ unchanged for every other principal. *Global erasure*, the right to be forgotten, is the special case that revokes p for all principals and additionally requires its bytes to be destroyed. The defining difficulty of revocation, absent from single-tenant deletion, is that p must vanish from P_u yet stay fully functional in every $P_{u'}$ that still contains it: p cannot be removed from G.

2.4 Threat model and requirements

We require revocation to take effect against three parties.

(1) Revoked principal. After revoke(u, p), principal u keeps issuing ordinary queries and observing their results. We treat u as honest-but-curious: it follows the protocol but may inspect what it receives. The requirement is that, for every query u issues after revocation, p neither appears in u's results nor influences which other vectors do; that is, u's search behaves as if p had never been in P_u. We treat this exposure structurally: because it constrains the index itself rather than any particular query, this requirement holds for adversarially crafted queries and under collusion among revoked principals, each of whose view independently excludes p.

(2) Honest-but-curious operator. For globally erased vectors, an operator with read access to process memory and to serialized snapshots must not be able to recover the erased bytes. We assume keys can be destroyed securely, for example inside a trusted execution environment [59]; given that, cryptographic shredding leaves only ciphertext that cannot be read.

(3) Auditor. A regulator or compliance auditor is not an adversary but a stakeholder who must receive verifiable evidence that a revocation or erasure was requested and took effect, which motivates an append-only, tamper-evident record.

We assume the index server faithfully enforces the current policy during traversal, exactly the enforcement we design; compromise of this reference monitor, timing and other side channels, and inference attacks on the embedding geometry are out of scope. These parties map directly onto the axes of Fig. 1: results, routing, and view scope for the revoked principal; physical erasure for the operator; verifiability for the auditor; and immediacy, the requirement that no exploitable window separate a revocation from its effect, across all three.

3 THE ZOMBIE VECTOR PHENOMENON

We now show, by direct measurement on a graph index, that lazy deletion fails in three concrete ways once it is asked to carry the weight of revocation. The setup is a clustered HNSW index on which vectors are revoked by whole semantic cluster, the realistic case in which permissions follow content (a department, a care unit, a clearance level). All numbers below are measured; the experimental protocol is in §7.

3.1 Revoked vectors are physically retained

A tombstone is a flag, not an erasure. On a million-scale clustered index, marking a vector deleted leaves the element count unchanged: the vector keeps its slot. Its embedding is then recoverable in two ways an operator or auditor would care about. First, the revoked embedding appears *verbatim* in the serialized index written to disk, so anyone with read access to the file or a backup obtains it directly. Second, revocation is reversible: clearing the tombstone returns the embedding byte-for-byte, even after the index has been saved and reloaded. Logical deletion thus erases nothing physical: the bytes persist until a consolidation pass overwrites them.

3.2 Zombie vectors steer authorized search

Filtering a vector from the answer does not filter it from the graph. The bridge b of Fig. 2 is the single-query picture: once revoked, b never appears in R_k, yet authorized search still routes through it to reach the vectors beyond. Across a workload this distortion accumulates as *drift*. To measure it we compare a user's authorized top-10 under Tombstone against an *oracle* index rebuilt on the authorized vectors alone, the correct per-view answer; drift is the fraction of that top-10 which differs from the oracle. Fig. 3(a) shows it rising with the revoked fraction, into the high teens of percent by half the index revoked. The user receives different answers because of data they are no longer permitted to see.

3.3 The cost of true erasure, and the compliance window

There is no cheap way to stop a zombie from routing. The only operation that removes a revoked vector's influence on traversal is the deferred consolidation pass, whose limiting case is a full rebuild, work that scales with the whole corpus rather than the one vector deleted. Fig. 3(b) shows its cost growing to about 93 s at eight million vectors, so paying it on every revocation is infeasible and systems must batch. Batching creates a *compliance window*: the longest time a revoked vector may keep routing before the next pass clears it. Fig. 3(c) makes the trap explicit. Bounding the window to one minute forces the index to spend most of its time rebuilding, and at eight million vectors a single rebuild no longer even fits inside the window; relaxing the window to one hour drops this under 3% but lets every revoked vector steer queries for that hour. Cost and retention are inversely coupled: under rebuild-based erasure, one is small only when the other is large.

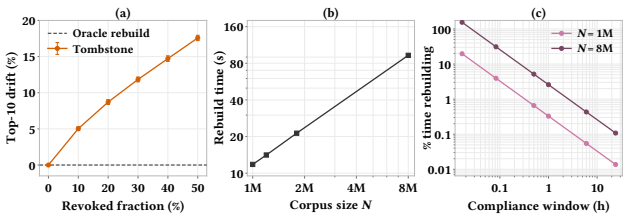

Figure 3: The deletion dilemma. *(a)* A revoked vector keeps steering authorized search: a user's top-10 *drift*, the fraction differing from an oracle per-view rebuild, grows with the revoked fraction even though revoked vectors never appear in the results. *(b)* Truly removing a vector and repairing navigability costs a rebuild that grows with corpus size; *(c)* bounding the compliance window, how long a revoked vector may still route, forces a large fraction of machine time into rebuilding, so cost and retention are inversely coupled. Clustered HNSW on the four datasets (1.0–8.0M indexed vectors); *(a)* means over ten seeds, bars one standard deviation; Recall@10 = 0.98 is the clean oracle index, while Tombstone falls to about 0.94 after revocation as routing distortion sets in; *(b,c)* rebuild cost is the full per-view rebuild; at eight million vectors the graph-build phase alone is 66.8 s while the full per-view rebuild reaches 93.0 s, means over ten seeds.

These are not three problems but one. Tombstone takes a vector out of the answer but leaves it in the graph, so it still persists and still routes, and the only operation that removes it rebuilds the whole index. Erasure under revocation therefore demands what lazy deletion cannot give: a vector gone from results, from routing, and from storage, immediately, for one view alone, and verifiably. §4 formalizes this standard; against it, no existing method stands.

4 FORMALIZING PER-VIEW ERASURE

To rule out those failures we need a precise target: what should it mean to erase a vector from one principal's view while it stays live for every other? We answer with six properties, conjoin them into a single guarantee, and prove no existing maintenance primitive meets it.

4.1 Erasure requests and the reference view

We reuse the notation of §2. An index $\mathcal{I} = (G, S)$ pairs a navigable graph $G = (P, E)$ over the stored vectors P with a physical store S holding their bytes. A query for principal u returns $\mathrm{Ans}(\mathcal{I}, q, u)$, the returned top-$k$ over u's view (the principal-aware form of R_k from §2), and its greedy traversal expands a set of nodes $\mathrm{Visit}(\mathcal{I}, q, u) \subseteq P$ (the principal-aware form of $\mathrm{Visit}(q, G)$, now ranging over the full index $\mathcal{I}$ and principal u). The view of u is $P_u = \{p \in P : u \in A(p)\}$.

An *erasure request* $\rho = (p, R)$ names a vector p and a scope $R \subseteq U$ of principals who must lose access to it. Revoking a single principal is $R = \{u\}$; global erasure is $R = U$. After ρ is applied, $u \notin A(p)$ for every $u \in R$, so $p \notin P_u$.

Table 1: The six axes of per-view erasure (Definition 4.1): each names a way a revoked vector p can persist after a request $\rho = (p, R)$ transforms the index $\mathcal{I}$ into $\mathcal{I}'$ with store S'. Ans and Visit are the answer and visited sets of §2; R=U is global erasure.

Axis	Condition on $\mathcal{I}'$
Results	$p \notin \mathrm{Ans}(\mathcal{I}', q, u)$ for every $u \in R$ and query q
Routing	$p \notin \mathrm{Visit}(\mathcal{I}', q, u)$: p is never distance-scored, inserted into the frontier, expanded, or returned [a]
Physical	if R=U, p's bytes are unrecoverable from S', in memory and every serialized snapshot
Immediate	the conditions above hold from the moment ρ is accepted
Verifiable	a tamper-evident record lets a third party confirm ρ took effect
View-scope	every $u' \notin R$ with $p \in P_{u'}$ keeps access to p and its recall [b]

[a] p's identifier may still be *encountered* in an adjacency list the search scans, since p stays a physical node shared with views that still hold it; the axis forbids p's *participation*, not its appearance in an edge list. [b] Not bit-for-bit identical: §7 measures the small top-k drift on unaffected views.

To say what u *should* observe once p leaves its view, we fix a reference: $\mathcal{I}_u^\star$, the index built on exactly P_u. This is the oracle per-view rebuild of §3: the behavior an index would have if p, and every other vector u cannot see, had never been inserted for u.

4.2 The six axes

A mechanism that answers $\rho = (p, R)$ by transforming $\mathcal{I}$ into $\mathcal{I}'$ may satisfy the six properties of Table 1. Routing is strictly stronger than Results: a method may hide p from the answer yet still let it bend the path that selects the other returned vectors, the zombie effect measured in §3. We keep the two separate because existing methods achieve one without the other.

Definition 4.1 (Per-view erasure). A mechanism achieves *per-view erasure* if, for every request $\rho = (p, R)$, the resulting index satisfies Results, Routing, Immediate, Verifiable, and View-scope for the scope R, and additionally Physical whenever $R = U$.

The reference $\mathcal{I}_u^\star$ is an empirical quality target, not part of the definition: per-view erasure pins down p's operational non-participation exactly, while §7 shows the resulting answers track $\mathcal{I}_u^\star$ to within its own seed-to-seed variation, rather than claiming identity with it.

We call the quantity Definition 4.1 drives to zero the *per-view leakage* of p into a revoked view: any way in which p still reaches u after revocation, whether through a returned result or through a traversal it steers. This is a structural property of the index, separate from inference about hidden data, which we do not study.

4.3 No existing primitive suffices

Theorem 4.2. *Each of Tombstone, Physical-delete, Periodic-rebuild, and cryptographic shredding violates at least two of the six axes. Hence none of them, applied alone, achieves per-view erasure.*

Proof. For each primitive we exhibit an axis it cannot satisfy; Fig. 1 records the full pattern.

Tombstone flags p deleted and filters it from answers (Results) instantly (Immediate), but leaves p a node of G with its edges intact.

It therefore keeps steering traversal (Routing fails; §3 measures the drift) and keeps its bytes in memory and snapshots (Physical fails). A mutable flag carries no proof of erasure (Verifiable fails), and it removes p for all principals at once rather than for a chosen view (View-scope fails).

Physical-delete removes p's node and repairs its in-neighbors, so p neither appears in nor routes any answer (Results, Routing) and its bytes are gone from the live store (Physical). But it acts on every principal (View-scope fails) and produces no auditable record (Verifiable fails).

Periodic-rebuild reconstructs the graph on the surviving set, satisfying Results and Routing once it runs. But it runs only at the next scheduled pass: until then p survives in the live store, so its bytes stay recoverable and the compliance window of §3 stays open on the physical axis as well (Physical and Immediate fail). It is also global (View-scope fails) and unaudited (Verifiable fails).

Cryptographic shredding destroys p's key, rendering its bytes unreadable (Physical) at once (Immediate) and checkably so (Verifiable). But it changes the bytes, not the graph: p's node and edges remain and no predicate excludes them, so the search still scores, expands, and can return p (Routing, Results fail); the bytes are merely unreadable, so the distance p contributes is meaningless rather than absent. And a key is bound to a record, not to a view (View-scope fails).

The View-scope column is thus empty for every prior primitive, and no single primitive covers Routing, Physical, Immediate, and View-scope together. □

The proof points to what a solution must combine: view-scoped control of both results and traversal, global byte destruction when the scope is everyone, and a single operation that is immediate and leaves an auditable trace. §5 builds exactly this. Appendix A gives the full formal model and the complete proofs.

5 LETHE: DESIGN

§4 fixed the target. Per-view erasure reads like a demand for a separate index per view; Lethe meets it over *one* shared structure, augmenting a standard navigable graph (HNSW or DiskANN) with four cooperating elements, each closing one or more axes of Definition 4.1.

5.1 Overview

Lethe keeps the base graph $G = (P, E)$ intact and adds four elements:

(1) a per-vector *access predicate* consulted during search, which makes "p is transparent for view u" an invariant of traversal (Results, Routing, Immediate, View-scope);
(2) an *effective-view-shared bypass overlay* that restores the navigability a revoked vector used to provide, without materializing a separate graph per user (preserving recall, hence View-scope for authorized principals);
(3) *vector-granular cryptographic shredding* that destroys a vector's bytes once its scope reaches everyone (Physical);
(4) an *append-only erasure log* that records every revocation and shred in tamper-evident form (Verifiable).

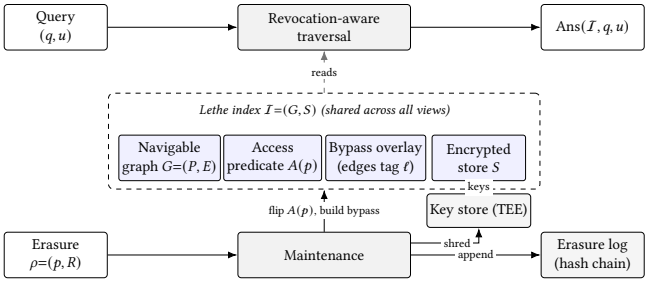

Figure 4: Lethe over one shared index, with the query path (top) and the erasure path (bottom) mirrored. A query (q, u) is answered by revocation-aware traversal, which reads the graph G, the access predicate $A(p)$, and the bypass overlay. An erasure request $\rho = (p, R)$ drives maintenance that flips $A(p)$ and builds bypass edges (Alg. 1), appends to the erasure log, and shreds the vector's key in the key store, which holds the per-vector keys that decrypt the encrypted store S; destroying a key renders that vector's bytes unreadable. Dashed edges are reads, solid edges are writes.

A revocation never rebuilds or repairs the graph eagerly, so it takes effect on the very next query. Figure 4 traces how a query and an erasure request move through these elements over the one shared index.

5.2 Revocation-aware traversal

Lethe attaches to each vector p its access set $A(p)$, stored compactly as a bitmap over the roles of the permission lattice L (§2), together with a small per-principal exception set recording individual grants or revocations layered on the role grant, so a revocation can target a whole role or a single principal; the test $u \in A(p)$ is $O(1)$. Greedy search proceeds as usual with one change at every expanded node p: if $u \notin A(p)$, then p is *transparent* to u. A transparent node is never entered into the result candidate set and never ranked, so it can neither appear in nor reorder u's answer (Results); and its distance never enters the frontier that selects the next node to expand, so it exerts no routing influence (Routing). Reachability past p is preserved by the overlay below, not by following p's own edges. Because the test reads live metadata, removing u from $A(p)$ takes effect on the next query with no structural change (Immediate) and alters nothing for any u' that still holds p (View-scope).

5.3 The effective-view-shared bypass overlay

Making p transparent removes a connector: p may have been the only short link between two regions of u's authorized subgraph, and dropping it is exactly the navigability-corrupting ghost of §3. Repairing this separately for each principal would rebuild a graph per user, the memory blow-up Lethe must avoid.

The repair, however, depends not on the individual principal but on the *effective view*: the view P_u of §2, the vectors u may actually see once role grants and any individual exceptions are applied. Principals with the same effective view need the same bypass around p. Under pure role-based access these classes coincide with the lattice elements, and the bypass is computed once per role; an individual

revocation that splits a role refines it into separate effective-view classes, leaving a still-authorized principal of that role undisturbed. Lethe therefore keys the bypass by effective-view class, written ℓ below, not by user.

Alg. 1 builds it. Let Prune$(x, cand, M)$ denote the base index's own neighbor-selection heuristic, which returns at most M close yet diverse neighbors of x from a candidate set *cand* (RobustPrune in DiskANN [47], the analogous heuristic in HNSW [34]), with M the out-degree bound of §2. When p becomes transparent across an effective-view class ℓ, the survivors it used to reach are its authorized out-neighbors $C = N_{\text{out}}(p) \cap P_\ell$. For each authorized in-neighbor $a \in N_{\text{in}}(p) \cap P_\ell$, which previously routed through p to reach C, Lethe re-selects a's neighbors over its current ones together with C using Prune, and records the newly chosen edges in the overlay, tagged ℓ. This is the same local repair a physical deletion would apply, but confined to P_ℓ and written to the overlay rather than into G.

Algorithm 1 Bypass construction for a transparent vector

Require: vector p, transparent across effective-view class ℓ; base graph G; out-degree bound M; pruning heuristic Prune
Ensure: overlay edges, tagged ℓ, restoring navigability of P_ℓ around p

```
C ← N_out(p) ∩ P_ℓ                ▷ authorized survivors reached via p
for each a ∈ N_in(p) ∩ P_ℓ do ▷ authorized in-neighbors routed via p
    T ← Prune(a, (N_out(a) ∪ C) ∩ P_ℓ, M)
    for each b ∈ T \ N_out(a) do
        add overlay edge a → b tagged ℓ
    end for
end for
```

Each construction touches only p's neighborhood: it prunes $|N_{\text{in}}(p) \cap P_\ell|$ in-neighbors over $O(M)$ candidates each, for $O(|N_{\text{in}}(p)| \cdot M)$ distance computations and no global rebuild. It runs once per (p, ℓ) and is reused by every principal in effective-view class ℓ that lost p. When p is transparent to u, the query follows the base edges of G together with the bypass edges tagged with u's effective-view class, whose endpoints lie in P_u and are therefore all visible to u. The construction does not recreate the edges a rebuild on P_ℓ would produce, overlapping them only partly (§7); it restores navigability instead, so greedy search reaches the authorized neighborhood nearly as well as on a rebuild, up to the pruning approximation. The overlay's size scales with the number of distinct (p, ℓ) repairs, bounded by the number of effective-view classes: the lattice elements under pure role-based access, refining only where individual exceptions split a role. §7 shows it stays far below a per-user index, rising from 1.24× to 1.29× the base graph as the exception rate grows from zero to one half, while holding recall close to $\mathcal{I}_u^\star$.

5.4 Vector-granular cryptographic shredding

Transparency hides p but does not destroy it. That is correct for revocation, where others still need p, but insufficient when the scope is everyone. Lethe stores each vector's payload encrypted under a per-vector key, with keys kept outside the index in a protected key store such as a trusted execution environment, following MemTrust [59]. Erasing p for all principals destroys its key: the ciphertext that remains in memory and in every snapshot can no longer be read (Physical); the destruction is instantaneous (Immediate); and destroying a named key is itself the evidence an auditor checks. The unreadable node is then made transparent to every view through the same predicate, with bypass edges installed at the top of the lattice, so global erasure reuses the traversal and overlay for results and routing and adds only byte destruction.

5.5 The append-only erasure log

Verifiability must not rest on the operator's word. Lethe records every request $\rho = (p, R)$ as an entry in an append-only, hash-chained log: the scope, a timestamp, and a commitment to the affected vector, signed so that entries cannot be altered or reordered without detection. An auditor checks two facts without trusting the operator: that the log is intact as an append-only chain, and that the live index agrees with it, namely that p's predicate excludes exactly R and, for a global erasure, that p's key is gone. Each erasure thus becomes a checkable fact (Verifiable).

5.6 Composition

A revocation alters only p's neighborhood, at constant or amortized cost and without a rebuild; §6 gives the procedure and its complexity. The four mechanisms compose to the full guarantee:

Proposition 5.1. *Lethe achieves per-view erasure (Definition 4.1): revocation-aware traversal secures Results, Routing, Immediate, and the traversal half of View-scope; the effective-view-shared bypass overlay secures the recall half of View-scope; the append-only erasure log secures Verifiable; and vector-granular cryptographic shredding secures Physical when $R = U$.*

The per-mechanism arguments are the four subsections above; the navigability of the bypass overlay, that it preserves authorized recall while removing the revoked node from every revoked view's reachable set, is proved in the extended version. Lethe thus fills the row left empty in Fig. 1, over one structure shared across views.

6 QUERY PROCESSING AND MAINTENANCE

§5 fixed the mechanisms; this section gives the search and maintenance procedures over them, with their costs.

6.1 Revocation-aware search

Alg. 2 answers a query q for principal u. It is the greedy best-first search of §2 with two changes, both keyed to u's effective-view class $\text{ev}(u)$, its role $\lambda(u)$ when no individual exception applies. First, a visited node b enters the frontier and the answer only when $u \in A(b)$; a transparent node ($u \notin A(b)$) is skipped, so it is neither ranked nor returned (Results) and its distance never steers the search (Routing). Second, when expanding a node a the search follows a's base out-edges together with a's bypass edges tagged $\text{ev}(u)$, written $\text{Bypass}_{\text{ev}(u)}(a)$; these detour around the transparent nodes a used to reach (§5). We write ef for the frontier width and take the entry point ep to be a navigation node, visible to every role and never a revocation target.

Algorithm 2 Revocation-aware search

Require: query q; principal u; entry point ep; frontier width ef; result size k
Ensure: the k vectors of P_u nearest to q

$W \leftarrow \{ep\}$ ▷ frontier of authorized candidates
while W has an unexpanded node **do**
  $a \leftarrow$ unexpanded node of W nearest to q; mark a expanded
  for each $b \in N_{\text{out}}(a) \cup \text{Bypass}_{\text{ev}(u)}(a)$ **do**
    if b not yet visited **then**
      mark b visited
      if $u \in A(b)$ **then** ▷ transparent nodes are skipped
        $W \leftarrow W \cup \{b\}$; keep the ef nodes of W nearest to q
      end if
    end if
  end for
end while
return the k nodes of W nearest to q

The access test $u \in A(b)$ is $O(1)$ (§6.3). The traversal otherwise visits the region a base search over P_u would, because $\text{Bypass}_{\text{ev}(u)}$ restores the reachability a rebuild on P_u would have (§5) and its out-degree is bounded like the base graph's by M. Search therefore keeps the base greedy-search cost, $O(ef \cdot M \cdot d)$ distance work along the walk for d-dimensional vectors, plus an $O(1)$ test per visited node. Recall matches the oracle per-view rebuild up to the pruning approximation of §5, which §7 measures.

6.2 Maintenance

Alg. 3 applies an erasure request $\rho = (p, R)$. It removes R from $A(p)$, a constant-time predicate update; for each effective-view class ℓ whose view P_ℓ loses p it ensures the bypass edges of Alg. 1 exist, building them once and sharing them across that class's principals; for a global request ($R = U$) it destroys p's key; and it appends one entry to the erasure log. No step rebuilds the graph.

Algorithm 3 Maintenance for an erasure request

Require: request $\rho = (p, R)$; key store; append-only erasure log

$A(p) \leftarrow A(p) \setminus R$ ▷ $O(1)$ predicate update
for each effective-view class $\ell \in \mathcal{L}(R)$ **do** ▷ classes whose view P_ℓ loses p
  if the bypass around p tagged ℓ is not yet built **then**
    run Alg. 1 on (p, ℓ)
  end if
end for
if $R = U$ **then**
  destroy p's key in the key store ▷ cryptographic shredding
end if
append $\langle p,\ R,\ \text{timestamp},\ \text{commit}(p)\rangle$ to the erasure log

Let $\mathcal{L}(R)$ be the set of effective-view classes whose view loses p under ρ: one class for a single-role revocation, $\text{ev}(u)$ for a single-principal revocation $R = \{u\}$, and every class whose view contained p for a global request $R = U$. The predicate update is $O(1)$; each bypass build is $O(|N_{\text{in}}(p)| \cdot M)$ (Alg. 1) and runs once per (p, ℓ); key destruction and the signed, hash-chained log append are $O(1)$. Maintenance therefore costs $O(|\mathcal{L}(R)| \cdot |N_{\text{in}}(p)| \cdot M)$, local to p's neighborhood, against the $\Theta(N)$ of a consolidation pass over N vectors. It takes effect on the next query.

Concurrency and crash recovery turn on one rule: a revocation linearizes at the predicate commit. Once $A(p) \setminus R$ is published, every query is denied p on Results and Routing, while the bypass build and log append may lag, as they bear on recall and audit, not reachability. After a crash, recovery restores the committed predicate and, for $R = U$, confirms the key is destroyed, so a revoked vector never reappears; §7 reports zero leakage after recovery, against the residue from methods recovering to a stale graph; Appendix D.3 drives a crash at each point of the maintenance and recovery path and confirms a zero-leakage safe state from every one.

6.3 Data structures and space

Each vector p carries its role tag $\lambda(p)$ and a short exception list of principals revoked from the role-induced default; the test $u \in A(p)$ then checks $\lambda(p) \sqsubseteq \lambda(u)$ against a precomputed reachability bitmap over the lattice and scans the exception list, in $O(1)$. The bypass overlay is stored as, for each node a, its bypass edges grouped by tag ℓ, so that $\text{Bypass}_{\text{ev}(u)}(a)$ is a single lookup. Its total size is $\sum_{(p,\ell)} |N_{\text{in}}(p) \cap P_\ell| \cdot M$ over the built repairs, bounded by the number of (vector, class) repairs rather than by the user population, and far below the $\Theta(|L| \cdot N \cdot M)$ of materializing one graph per role. Building those edges needs p's in-neighbors $N_{\text{in}}(p)$, which a graph storing only out-edges does not provide; Lethe keeps a reverse-edge directory, updated on insertion, that returns $N_{\text{in}}(p)$ in a few milliseconds by lookup, an order of magnitude under the full-graph scan the alternative would require, with a high in-degree hub the worst case (§7). The erasure log is an append-only hash chain holding one signed record per request, $O(|\text{requests}|)$ in size. The base graph G and the encrypted store S are untouched by revocation, so a deployment pays only for the predicate bits, the reverse-edge directory, the overlay it actually uses, and the log.

7 EVALUATION

Our evaluation establishes four claims, in the order a skeptical reader would test them. First, the routing channel is real: a revoked vector that has been filtered from every result still steers the queries of the very view it was revoked from, and the effect is large enough to measure and to exploit. Second, result filtering is therefore not erasure, because the channel it leaves open is the one a graph index depends on to navigate. Third, Lethe closes both the routing channel and the physical residue while holding recall at the level of an oracle per-view rebuild and leaving unaffected views untouched. Fourth, it does so at a cost that is acceptable against the three alternatives a practitioner would otherwise reach for: rebuilding the index, materializing one index per view, and filtering search. We take these in turn after fixing the setup.

7.1 Setup

Datasets. We use four corpora chosen to separate a graph-index property from a text-encoder artifact. Three are access-controlled text collections: a legal corpus built on LegalBench [24] in a retrieval-augmented configuration [2], an open-domain web collection from

MS MARCO [10] under a synthesized role hierarchy, and MIMIC-IV [30, 31] clinical notes as a sensitive-domain case study [45]. The fourth, SIFT1M [28], carries exact ground truth and no text encoder, so the zombie effect appearing there shows it is a property of graph navigation rather than of any embedding model. Text corpora are embedded once with BGE-large-en-v1.5 [49] at 1024 dimensions and cached, so every method reads identical vectors.

Permission workload. On top of each corpus we generate SHARED-CONFIDENTIAL, an overlapping role-based access structure in which each vector belongs to one or more roles, 80% of vectors are shared across several roles and 20% are role-private, and role popularity follows a Zipfian distribution. Users are assigned to roles, and a revocation withdraws a subset of the shared vectors from one user or role while every other principal that holds them stays authorized. We sweep the revoked fraction from 0 to 0.5 and the revocation pattern over random, hub, and cluster. Cluster is the primary pattern: permissions in practice track semantic groups, a document class or a care unit, so a revocation removes a coherent region of the embedding space rather than scattered points, the regime in which a graph index is hardest to keep navigable and that most stresses any erasure mechanism. Sensitivity to the predicate exception rate, to user-level revocation within a shared role, and to view distance from the revoked principal is reported in Appendices D.4, D.5, and D.6.

Index and operating point. All methods index the same vectors with HNSW [34] through hnswlib at top-k with k=10, out-degree M=16, and `ef_construction`=200. We fix a single `ef_search` across all methods and revoked fractions, tuned so the oracle per-view rebuild reaches Recall@10 $\approx$ 0.98 on the authorized set; the methods therefore share one search budget, and the differences in Recall@10 reported in Section 7.3 come from how each method indexes and filters rather than from a different operating point. The characterization runs use single-threaded, fully seeded builds, which make the graph, and therefore the drift it produces, reproducible across machines; throughput, scalability, and concurrency runs use multiple threads and are reported as trends. Exact ground truth is computed by blocked ℓ_2 search. Generality to a second graph index, DiskANN [47], and to a second encoder is reported in Appendix D.2 (Table 14).

Baselines. We compare against the alternatives a practitioner can deploy today, all on the same vectors, search budget, and workload. *Tombstone* marks a revoked vector deleted and filters it from results while leaving it in the graph, the production default [44]. *Post-filter* enforces access by row-level security over a shared index [42]. *Physical-delete* removes the vector and repairs only the affected neighborhoods, and *Periodic-rebuild* consolidates the whole index on a schedule [44]. *Per-role-index* materializes a separate index per role, the cost upper bound a per-view guarantee naively implies [29, 57]. *ACORN* [39] and *HoneyBee* [57] are predicate-filtered and role-partitioned access-controlled indexes, and *SPFresh* [52] is a streaming in-place maintenance system. *Oracle* rebuilds an index on exactly the authorized vectors of each view; it is the gold standard for what an erased index should return, and we use it as the reference for drift and recall rather than as a deployable baseline.

Metrics. Every top-10 list below is restricted to authorized vectors, so a revoked vector can never appear in a result and any divergence we report comes from routing rather than from a leaked answer. *Result leakage* is the fraction of a returned top-10 that are revoked vectors, and it is zero by construction for every method that filters results. *Top-10 drift* is one minus the average overlap between a method's authorized top-10 and the top-10 of the *Oracle* per-view rebuild, taken over the query workload; it measures how far routing through revoked vectors bends the answer away from a truly erased index. *Operational leakage* instruments the search loop and reports the fraction of traversal events, namely distance evaluations, frontier insertions, and expansions, that touch a revoked vector; it is the routing component of *per-view leakage* (§4) and measures the zombie mechanism directly, so drift cannot be dismissed as approximation noise. *Recall@10* is the overlap of the authorized top-10 with the exact ten nearest authorized neighbors. To ground leakage in an adversary's success we add two attacks: a *distinguishing* test reports the AUC of the strongest classifier that decides, from observed outputs, whether a revoked vector still influences a view, where 0.5 means indistinguishable from the *Oracle*; and a *reconstruction* attack runs embedding inversion [37] on bytes recovered from a serialized snapshot and reports token F1 and exact match. Cost is measured by revocation throughput, the compliance window (§3), memory relative to per-user and per-role materialization, and a per-query latency breakdown.

Protocol. The headline results, the existence of the channel, operational leakage, recall, and view isolation, use ten seeds; ablation, memory, and compliance-window results use five. We report the mean and standard deviation over seeds, the bootstrap 95% confidence interval of the gap to LETHE (10^5 resamples, stratified by dataset and revocation pattern), a paired bootstrap significance test with Holm-Bonferroni correction across the baselines in each table, and a rank-biserial effect size. We use a bootstrap rather than a signed-rank test because, when one method dominates another on every paired run, a rank test is bounded by its own discrete resolution rather than by the evidence. Seeds, tests, and corrections are fixed before running. Machine-dependent quantities, latency, throughput, and scaling, are reported as trends with the machine stated and carry no significance tests, since their absolute values are not comparable across hardware. Appendix B gives the full datasets, build configuration, and artifact manifest.

7.2 A revoked vector still routes

Filtering a revoked vector out of the result is easy, and every method does it: result leakage is zero throughout (Fig. 5). The quantity that matters is what the search does on the way to that result. Figure 5(a) measures it by instrumenting the traversal. Under Tombstone, the production default, the fraction of traversal events that touch a revoked vector climbs with the revoked fraction, reaching 25.9% at one half; Post-filter is marginally worse, and a lazily repairing streaming index, SPFresh, still routes through revoked vectors at the same point. The cause is structural. All three leave the revoked vectors in the graph, so the walk keeps visiting them, expanding them, and scoring their neighbors on every query, for the very user from whom they were revoked.

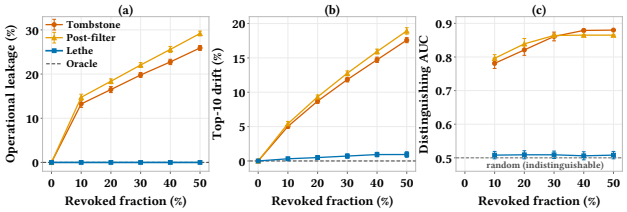

Figure 5: A revoked vector is gone from the answer and still steering the search. Result leakage is zero for every method, so a revoked vector never appears in a result and all divergence shown here is routing. *(a)* The fraction of traversal events that distance-score, insert into the frontier, or expand a revoked vector reaches 25.9% under Tombstone and 29.2% under Post-filter at half the index revoked, because both keep revoked vectors in the graph; Lethe encounters a revoked identifier but never scores, inserts, or expands it, so the value is identically zero and coincides with the oracle per-view rebuild. *(b)* The authorized top-10 consequently drifts from a truly erased index, by 17.6% (Tombstone) and 18.9% (Post-filter) at the same point, against 0.9% for Lethe. *(c)* A classifier given only a user's own outputs separates a tombstoned or post-filtered index from the oracle at AUC up to 0.88 and 0.86, while Lethe stays at 0.51, on the random line. SharedConfidential, cluster revocation, four datasets, ten seeds; bars are one standard deviation.

This traversal changes the answer. Figure 5(b) shows the authorized top-10 drifting from the result an index would return had the vectors truly been removed, by 17.6% under Tombstone at half the index revoked. Because revoked vectors are filtered from the result, every one of these differences is a reordering among *authorized* vectors. The revoked vectors are not appearing in the answer, they are deciding which permitted vectors do. The effect is largest under cluster revocation, where a withdrawn semantic region leaves a gap the walk must cross, the access pattern that arises whenever permissions track a document class or a care unit.

Whether a curious user can act on this is Fig. 5(c). A classifier given nothing but a user's own top-10 outputs separates a tombstoned index from an oracle per-view rebuild at an AUC of 0.88 at half revocation, with the advantage growing monotonically in how much has been revoked. The residual routing influence is therefore not only measurable by us under full instrumentation but recoverable by an adversary holding nothing but black-box access to its own results. The signal is the continued presence of the revoked vectors; an index that had erased them would offer nothing to separate.

The ordering holds on every dataset: Figure 6 breaks it down per collection, with Lethe holding drift to 0.7–1.2% across collections, matching the per-role index at a fraction of its memory, while Tombstone and Post-filter sit at 17–19%.

The vector persists in more than the traversal. The other half of the zombie definition is physical: can the revoked content be recovered from what the system still stores? Appendix F reports the inference attacks, adversarial revocation targeting, and audit tamper-evidence behind this. Table 2 pairs byte-exact recovery from

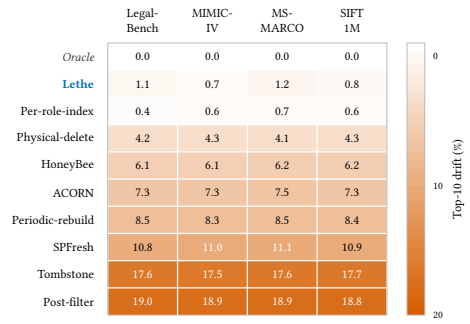

Figure 6: The zombie effect and Lethe's correction are consistent across datasets. Top-10 drift of the authorized answer from a per-view rebuild (%), per method and per dataset, at half the index revoked. The ordering and magnitudes are stable across all four collections: the oracle is exact, Lethe holds drift to 0.7–1.2% (matching the per-role index without its memory cost), while Tombstone and Post-filter drift by 17–19% on every dataset. SharedConfidential, cluster revocation, mean over ten seeds.

Table 2: Physical persistence of a globally erased vector: whether its bytes survive byte-exact in a serialized snapshot (*Byte-rec.*), and the token F1 of an embedding-inversion reconstruction from the recovered bytes (*Recon. F1*). SharedConfidential, four datasets; the reconstruction runs only on the three with a text encoder, mean over ten seeds with half the index revoked. The 0.01 floor is the reconstruction of a vector the system never stored.

Method	Byte-rec.	Recon. F1	Method	Byte-rec.	Recon. F1
Tombstone	✓	0.42	Periodic-rebuild	✓	0.42
Post-filter	✓	0.42	Physical-delete	✗	0.01
SPFresh	✓	0.41	Per-role-index	✗	0.01
ACORN	✓	0.42	Lethe	✗	0.01
HoneyBee	✓	0.42	Oracle	✗	0.01

a serialized snapshot with the embedding-inversion reconstruction run on whatever bytes that recovery yields. Every method that leaves the vector's bytes on disk, including the access-control overlays of ACORN and HoneyBee, keeps it byte-exact in the snapshot and reconstructable at token F1 0.42 with half the index revoked; only the methods that remove the bytes, Lethe's cryptographic shredding among them, drop reconstruction to 0.01, the floor for a vector that was never stored. Tombstone misses the physical axis as squarely as it misses routing: the revoked embedding sits one snapshot read and one inversion away.

Lethe behaves throughout like the oracle, not the production default. Its operational leakage is identically zero, because revocation-aware traversal makes a revoked vector unreachable for the view, not merely absent from the result; its drift is 0.9% at half revocation, an order of magnitude below Tombstone; and its distinguishing AUC is 0.51, on the random line. The gap between Lethe and methods that keep the vector is consistent across all four datasets and

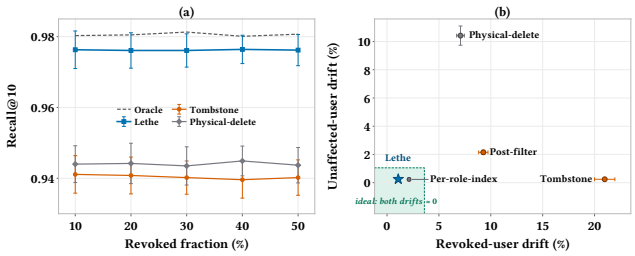

Figure 7: Lethe reaches the answer quality of an oracle per-view rebuild without its cost, and changes only the view requesting the revocation. *(a)* Recall@10 against revoked fraction. Lethe holds 0.976, just under half a point below the oracle and more than three points above Physical-delete (0.944) and Tombstone (0.940), which repair only locally or not at all and lose navigability. *(b)* Each method placed by the revoked user's drift from the answer it should now receive (x) and an unaffected user's drift from the answer it should still receive (y), both against a per-view rebuild; the origin is ideal. Tombstone and Post-filter spare unaffected users but fail the revoked one (21.0% and 9.3%, the zombie effect); Physical-delete erases but restructures the shared index and so disturbs still-authorized users, raising the view-isolation violation rate to 39.0%. Only Lethe and the per-role index reach the corner. SharedConfidential, cluster revocation, four datasets; panel (b) at 80% shared vectors, high role overlap, and 100 roles, and error bars are one standard deviation.

every seed: at half revocation Lethe is better on every paired run, so the rank-biserial effect size is −1.0 on leakage and drift and +1.0 on recall, the 95% bootstrap confidence interval of each gap excludes zero, and a paired bootstrap test is significant at $p \leq 0.001$ on every dataset after Holm correction. The full per-cell statistics for all 48 comparisons appear in Appendix G. The result is the one filtering cannot reach: a revoked vector is gone from the answer under every method, yet still steers the search; and once globally erased, its bytes stay recoverable on disk under every method keeping the shared graph navigable, Lethe alone excepted.

7.3 Erasure without losing recall or disturbing other views

A per-view guarantee is worthless if it degrades the answer it protects, and there are two ways it could. The authorized user could lose recall, or the residual drift we just measured could be uncaught leakage rather than benign approximation error. Figure 7(a) settles the first. Lethe holds Recall@10 at 0.976 across the revocation range, a hair below the oracle per-view rebuild and well clear of Physical-delete's recall cost ($p \leq 0.001$, paired bootstrap). Physical-delete is the cautionary case: removing the vector and repairing only its local neighborhood damages navigability and costs recall, exactly the expense the bypass overlay is designed to avoid.

The second concern is ruled out by decomposing the drift against the oracle into the part any approximate index incurs and the part that is genuine erasure residue. The approximation floor is fixed independently of Lethe: two independently seeded oracle rebuilds of the same authorized set drift 0.45% from each other. Lethe's 0.9% sits just above this floor, the gap being the bypass overlay's pruning approximation rather than erasure residue, so its answer is, to measurement, a per-view rebuild. Tombstone and Post-filter instead drift 17.6% and 18.9%, leaving 17.1 and 18.5 points that the floor cannot explain. That residue is the zombie routing of §7.2, now read as error in the delivered result rather than as traffic in the traversal.

Erasure must also stay local to the view that asked for it: revoking a vector from one user must not move what another user who still holds it retrieves. Figure 7(b) plots each method's two drifts against its own correct per-view rebuild, and they divide along exactly the axis the formal model predicts. Tombstone and Post-filter spare the unaffected user but abandon the revoked one; Physical-delete inverts this: it erases, but because it mutates the single shared index it disturbs users who still have access, pushing the view-isolation violation rate to 39.0%. Only Lethe and the per-role index hold both ends, keeping the unaffected user more than an order of magnitude tighter than Physical-delete ($p \leq 0.001$).

Table 3: End-to-end retrieval-augmented generation quality on LegalBench-RAG with a fixed open 7B generator (%). Methods that do not alter the returned set are omitted.

Method	Exact match	F1	Method	Exact match	F1
Post-filter	38.90	52.36	HoneyBee	41.31	54.34
Tombstone	39.40	52.76	Lethe	42.80	55.48
ACORN	40.90	53.84	Oracle	43.20	55.80

The leakage is not only geometric; it reaches the answer the user reads. Feeding each method's retrieved set to a fixed open 7B generator on LegalBench-RAG (Table 3) shows that routing through revoked vectors costs end-to-end quality: Tombstone and Post-filter lose more than three F1 points against an oracle per-view rebuild, while Lethe matches the oracle, 55.5 against 55.8. The zombie vectors do not merely change which authorized neighbors are retrieved; they degrade the answer generated from them.

Lethe thus reaches a corner the other deployable methods cannot hold at once: the recall and erasure quality of an oracle rebuild together with the view isolation of an index that was never restructured. The per-role index reaches the same corner and the oracle defines it; what does each pay to get there?

7.4 The cost of getting there

Table 4 and Fig. 8 show it: every baseline is pushed out of the corner along the axis it pays on. Appendix C gives the complete thirteen-method comparison across all six axes.The cheapest-looking methods never erase. Tombstone sustains 95,000 revocations per second, but its compliance window is 3,800 seconds, because nothing removes the vector from the graph until a full consolidation runs; its throughput is that of doing nothing. The methods that do erase pay for it, along opposite axes: Physical-delete holds the window near zero at the cost of throughput, while Periodic-rebuild buys throughput back only by leaving a window where the vector still routes.

Table 4: Cost of erasure. Revocations per second and the compliance window, the longest time a revoked vector keeps routing, are throughput measurements on one machine and are reported as trends. Memory is index size relative to the base graph. Filtering methods are cheap but never erase, so their window spans the consolidation interval; methods that do erase pay in throughput (Physical-delete, Periodic-rebuild) or in memory (the per-role index). LETHE pays neither. SHAREDCONFIDENTIAL, four datasets, mean over five seeds.

Method	Revocations per second	Compliance window (s)	Memory (×base)
Tombstone	95,077	3800.0	1.00
SPFresh	17,011	0.05	1.08
Physical-delete	2,461	0.05	0.96
Periodic-rebuild	10,931	651.6	1.06
Per-role-index	4,000	0.05	84.25
LETHE	49,975	0.004	1.26

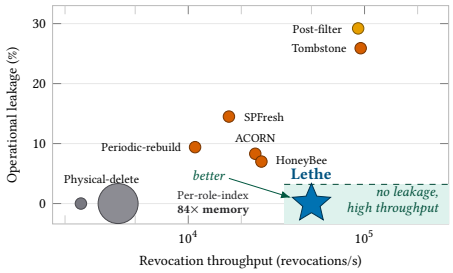

Figure 8: Throughput, leakage, and memory across erasure methods. Each method is placed by revocation throughput (x, log scale) and operational leakage at half revocation (y); marker size reflects index memory, with the per-role index drawn conspicuously large. Methods that retain the revoked vector are cheap but leak (upper region); methods that erase it pay in throughput (left) or in memory (the per-role index, 84× the base graph at 100 roles); only LETHE reaches the corner of no leakage at high throughput and near-base memory. The oracle per-view rebuild carries no steady-state revocation cost and is omitted. SHAREDCONFIDENTIAL, cluster revocation, four datasets; throughput and memory over five seeds, operational leakage as in §7.2 over ten seeds.

This is the compliance-window trap of §3 made concrete: with erasure bound to a structural pass, a short window costs throughput and high throughput costs a long window.

The per-role index escapes the trap by giving each role its own index, so a revocation is immediate, but it pays in memory: one materialized index per role costs 84 times the base graph at 100 roles and grows linearly in the number of roles, reaching 421 times at 500 roles. LETHE obtains the same per-view erasure for 1.26× the base graph, flat in the number of roles while the per-role index grows linearly, 1.5% of the per-role index at 100 roles (0.3% at 500) and 0.0014% of a per-user index, a materialization that at 100,000 users would cost 88,000× the base graph, because the bypass edges

Table 5: Ablation of LETHE. Removing a mechanism forfeits the one axis of Fig. 1 it secures and leaves the others intact: revocation-aware traversal secures routing, the bypass overlay secures navigability (recall), cryptographic shredding secures physical unrecoverability, and the erasure log secures verifiability. The last two rows isolate the design choice behind the memory result of §7.4; *per-user overlay* replaces the effective-view-shared overlay with one materialized per user. SHAREDCONFIDENTIAL, cluster revocation, four datasets, mean over five seeds. Column headers abbreviate operational leakage (*Oper. leak.*), physically unrecoverable (*Phys. unrec.*), and memory (*Mem.*); *Recall@10* and *Auditable* are line-wrapped, not abbreviated.

Configuration	Oper. leak. (%)	Recall @10	Phys. unrec.	Audit-able	Mem. (GB)
LETHE (full)	0.00	0.976	✓	✓	20.0
without revocation-aware traversal	24.81	0.955	✓	✓	20.0
without bypass overlay	0.00	0.910	✓	✓	19.4
without cryptographic shredding	0.00	0.976	✗	✓	20.0
without erasure log	0.00	0.976	✓	✗	20.0
without overlay pruning	0.00	0.977	✓	✓	21.2
per-user overlay	0.00	0.979	✓	✓	21.5

are shared across principals with the same effective view instead of being replicated per role or per user.

LETHE alone pays none of these. It erases immediately, with a 0.004-second window that is five to six orders of magnitude below the consolidation- and rebuild-based methods; it sustains 50,000 revocations per second, the highest of any method that actually erases; and it does so at base-index memory. The query path absorbs the predicate check and overlay lookup at a 95th-percentile latency of 16.0 ms, fourteen percent over Tombstone and below every method that restructures the graph. LETHE's five added query-path components total 1.1 ms over the base graph search, within twelve percent of Tombstone; the extended version decomposes the mean per-query latency per component. The design holds at scale. At one billion vectors a query returns in 7.8 ms and a revocation applies in 0.057 ms, the overlay adding 193 GB to a 3.0 TB index; the same near-oracle drift and zero operational leakage hold on DiskANN/Vamana and a second encoder (Table 14). The compliance-window trap is not a law of nature but a consequence of binding erasure to the index structure; move erasure into the traversal and the structure never has to be rebuilt, so immediacy costs neither throughput nor memory.

7.5 Each mechanism secures one axis

Table 5 removes each one in turn, and each forfeits exactly the axis it secures and no other: without revocation-aware traversal, operational leakage returns to 24.8% while recoverability and auditability are untouched; without the bypass overlay, leakage stays at zero but recall falls; without cryptographic shredding, the revoked bytes become recoverable from a snapshot; and without the erasure log,

no revocation can be proven after the fact. The six axes are thus a decomposition, not a list: no mechanism does two jobs and none is redundant. Appendix E extends the ablation with the overlay construction and scope variants.The last two rows show why Lethe shares the overlay across principals with the same effective view rather than materializing it per user: the shared overlay is small (0.6 GB, 3% of the index), but a per-user overlay roughly triples it and raises total memory by about 7%. Sharing is what keeps Lethe's per-view erasure at 1.26× the base graph (§7.4).

8 RELATED WORK

Streaming index maintenance. A large body of work keeps a graph or inverted index fresh under insertions and deletions. FreshDiskANN [44] and SPFresh [52] consolidate deletions in the background; incremental IVF maintenance [36], proximity-graph maintenance [53], real-time HNSW updates that repair unreachable points [50], in-place graph-index updates [51], and topology-aware localized updates [55] each reduce the recall and latency cost of churn. All score a deletion by whether recall recovers. None asks whether a deleted vector still steers traversal, whether its bytes survive, or whether the effect can be confined to one view, the questions a revocation raises and Lethe answers.

Access-controlled vector search. Filtering and partitioning systems enforce who may retrieve a vector. ACORN [39] and FilteredDiskANN [21] attach predicates to graph search; HoneyBee [57] and Curator [29] partition by role or tenant; production systems such as Milvus [48] and Manu [25] expose row-level security in the spirit of query rewriting [42], and zero-trust memory architectures [59] carry the model into agent memory. Every one enforces the results axis but not the routing or physical axes: §7.2 shows a revoked vector still steering traversal under ACORN and HoneyBee, as it must under any method that touches only the result. Lethe instead removes it from traversal and storage, the routing and physical axes a result-level filter leaves untouched. A complementary line protects the confidentiality of the query or data rather than controlling who may see a result. Private ANN search lets a client search without revealing its query to the server: PACMANN [58] retrieves graph neighborhoods by per-hop private information retrieval (PIR), Panther [33] combines PIR with secret sharing and homomorphic encryption in a single-server setting, while secure k-nearest-neighbor classification [43] and federated vector search across organizations [16, 56, 60] protect data split between parties. Each adds cryptographic or communication cost to keep the query or underlying data confidential, orthogonal to removing a revoked vector from a shared plaintext index.

Leakage channels and inference control. Restricting what a system returns has long been known to fall short of stopping disclosure. Statistical databases refusing to answer small query sets were still compromised by *trackers*, characteristic formulas that recover the forbidden statistics [12, 13], despite inference controls built to stop them [11]; multilevel and statistical databases leak through inference channels that combine constraints with auxiliary data [4, 9, 17, 27]; and encrypted or outsourced databases leak through access patterns and response volumes, which leakage-abuse and reconstruction attacks turn into recovered queries and records [7, 22, 23, 26, 32, 38]. The common thread is a side channel that outlives a result-level restriction. Operational leakage is this phenomenon inside a graph index: a revoked vector filtered from the answer still steers traversal, and the neighborhoods it shifts are a side channel an adversary reads off (§7.2). The stored bytes are not opaque either: embedding inversion reconstructs source text from a vector alone; Morris et al. [37] recover 92% of 32-token inputs exactly and extract full names from clinical notes given only vector-database access and embedding pairs. Lethe therefore treats leakage as a property of navigational structure and stored bytes rather than of the result set, closing the routing channel with revocation-aware traversal and the content channel with vector-granular cryptographic shredding, both held to per-view scope.

Secure deletion, unlearning, and the right to be forgotten. A separate line formalizes what deletion should mean. Secure-deletion systems erase bytes from storage media [41]; machine unlearning removes a training example's influence from a model [3, 6, 20]; the right to be forgotten has a cryptographic definition [18], and dependency-aware erasure asks what else must go when one record does [8], all driven by regulation [15]. Lethe carries these guarantees where they have not reached: into the navigational structure of a shared ANN index, where deletion must also stop routing, hold to per-view scope, and take effect at once.

Verifiable retrieval and inference privacy. V3DB proves vector-search results against a committed snapshot with zero-knowledge proofs [40]; Lethe's erasure log is a lighter append-only audit trail for revocations rather than a per-query proof, and the two compose. Distinct again is the line bounding what query answers reveal about the data through differential privacy over selection and top-k [14, 19, 35]; limiting inference about a present record is orthogonal to removing a revoked one from a shared index, the problem we take up.

9 CONCLUSION AND FUTURE WORK

A revoked vector filtered from every result can still steer the search that produces it, remain recoverable on disk, and, when finally removed, disturb users who never lost access. We named this the zombie vector phenomenon, formalized per-view erasure over six axes, and showed that no standard maintenance primitive achieves it. Lethe does, by moving erasure out of the index structure and into the traversal and the keys: revocation-aware traversal, an effective-view-shared bypass overlay, cryptographic shredding, and an append-only erasure log, each closing exactly one axis. Across four datasets, two graph indexes, and up to a billion vectors, it drives operational leakage to zero, holds recall within half a point of an oracle per-view rebuild, leaves unaffected views untouched, erases immediately, and costs 1.26× the base graph where a per-role index costs 84× at 100 roles.

Two assumptions bound these guarantees. Cryptographic shredding reduces physical erasure to key destruction, inheriting the trust placed in the key-management layer: a server that secretly retains key material defeats it, as it would any such scheme. And the memory advantage depends on permissions that overlap, since the bypass overlay is sized by the number of distinct effective views; a workload in which almost every vector is private to one principal

drives it toward the per-user cost it otherwise avoids. Timing results are single-machine trends, and we evaluate revocation rather than wholesale changes to the lattice, whose overlay maintenance we leave to future work.

The lesson outlives the system. In any shared structure that is navigated rather than scanned, removing a record from the answers is not the same as removing it from the system: the structure itself carries the record's influence, and an access check at the output cannot reach it. Erasure has to be a property of how the structure is traversed, not only of what it returns.

REFERENCES

[1] Maya Anderson, Guy Amit, and Abigail Goldsteen. 2025. Is My Data in Your Retrieval Database? Membership Inference Attacks Against Retrieval Augmented Generation. In *Proc. ICISSP'25*. SCITEPRESS, 474–485.

[2] Ryan Calvin Barron, Maksim Ekin Eren, Olga M. Serafimova, Cynthia Matuszek, and Boian S. Alexandrov. 2025. Bridging Legal Knowledge and AI: Retrieval-Augmented Generation with Vector Stores, Knowledge Graphs, and Hierarchical Non-negative Matrix Factorization. In *Proc. ICAIL'25*. ACM, 51–60.

[3] Lucas Bourtoule, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. Machine Unlearning. In *Proc. IEEE S&P'21*. IEEE, 141–159.

[4] Alexander Brodsky, Csilla Farkas, and Sushil Jajodia. 2000. Secure Databases: Constraints, Inference Channels, and Monitoring Disclosures. *IEEE Trans. Knowl. Data Eng.* 12, 6 (2000), 900–919.

[5] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6 (2024), 246:1–246:27.

[6] Yinzhi Cao and Junfeng Yang. 2015. Towards Making Systems Forget with Machine Unlearning. In *Proc. IEEE S&P'15*. IEEE Computer Society, 463–480.

[7] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. 2015. Leakage-Abuse Attacks Against Searchable Encryption. In *Proc. CCS'15*. ACM, 668–679.

[8] Vishal Chakraborty, Youri Kaminsky, Sharad Mehrotra, Felix Naumann, Faisal Nawab, Primal Pappachan, Mohammad Sadoghi, and Nalini Venkatasubramanian. 2025. Meaningful Data Erasure in the Presence of Dependencies. *Proc. VLDB Endow.* 18, 10 (2025), 3435–3448.

[9] Francis Y. L. Chin and Gultekin Özsoyoglu. 1982. Auditing and Inference Control in Statistical Databases. *IEEE Trans. Software Eng.* 8, 6 (1982), 574–582.

[10] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Jimmy Lin. 2021. MS MARCO: Benchmarking Ranking Models in the Large-Data Regime. In *Proc. SIGIR'21*. ACM, 1566–1576.

[11] Dorothy E. Denning. 1980. Secure Statistical Databases with Random Sample Queries. *ACM Trans. Database Syst.* 5, 3 (1980), 291–315.

[12] Dorothy E. Denning, Peter J. Denning, and Mayer D. Schwartz. 1979. The Tracker: A Threat to Statistical Database Security. *ACM Trans. Database Syst.* 4, 1 (1979), 76–96.

[13] Dorothy E. Denning and Jan Schlörer. 1980. A Fast Procedure for Finding a Tracker in a Statistical Database. *ACM Trans. Database Syst.* 5, 1 (1980), 88–102.

[14] David Durfee and Ryan M. Rogers. 2019. Practical Differentially Private Top-k Selection with Pay-what-you-get Composition. In *Advances of NeurIPS'19*. 3527–3537.

[15] European Parliament and Council of the European Union. 2016. Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, L119.

[16] Zeheng Fan, Yuxiang Zeng, Zhuanglin Zheng, and Yongxin Tong. 2025. FedVSE: A Privacy-Preserving and Efficient Vector Search Engine for Federated Databases. *Proc. VLDB Endow.* 18, 12 (2025), 5371–5374.

[17] Csilla Farkas and Sushil Jajodia. 2002. The Inference Problem: A Survey. *SIGKDD Explor.* 4, 2 (2002), 6–11.

[18] Sanjam Garg, Shafi Goldwasser, and Prashant Nalini Vasudevan. 2020. Formalizing Data Deletion in the Context of the Right to Be Forgotten. In *Advances of EUROCRYPT'20 (LNCS)*, Vol. 12106. Springer, 373–402.

[19] Jennifer Gillenwater, Matthew Joseph, Andres Muñoz Medina, and Mónica Ribero Diaz. 2022. A Joint Exponential Mechanism For Differentially Private Top-k. In *Proc. ICML'22 (Proceedings of Machine Learning Research)*, Vol. 162. PMLR, 7570–7582.

[20] Antonio Ginart, Melody Y. Guan, Gregory Valiant, and James Zou. 2019. Making AI Forget You: Data Deletion in Machine Learning. In *Advances of NeurIPS'19*. 3513–3526.

[21] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proc. WWW'23*. ACM, 3406–3416.

[22] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. 2018. Pump up the Volume: Practical Database Reconstruction from Volume Leakage on Range Queries. In *Proc. CCS'18*. ACM, 315–331.

[23] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. 2019. Learning to Reconstruct: Statistical Learning Theory and Encrypted Database Attacks. In *Proc. IEEE S&P'19*. IEEE, 1067–1083.

[24] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, K. Aditya, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel N. Rockmore, Diego Zambrano, Dmitry Talisman, Enam Hoque, Faiz Surani, Frank Fagan, Galit Sarfaty, Gregory M. Dickinson, Haggai Porat, Jason Hegland, Jessica Wu, Joe Nudell, Joel Niklaus, John J. Nay, Jonathan H. Choi, Kevin Tobia, Margaret Hagan, Megan Ma, Michael A. Livermore, Nikon Rasumov-Rahe, Nils Holzenberger, Noam Kolt, Peter Henderson, Sean Rehaag, Sharad Goel, Shang Gao, Spencer Williams, Sunny Gandhi, Tom Zur, Varun Iyer, and Zehua Li. 2023. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In *Advances of NeurIPS'23*.

[25] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. *Proc. VLDB Endow.* 15, 12 (2022), 3548–3561.

[26] Mohammad Saiful Islam, Mehmet Kuzu, and Murat Kantarcioglu. 2012. Access Pattern disclosure on Searchable Encryption: Ramification, Attack and Mitigation. In *Proc. NDSS'12*. The Internet Society.

[27] Sushil Jajodia and Catherine Meadows. 1995. Inference Problems in Multilevel Secure Database Management Systems. In *Information Security: An Integrated Collection of Essays*. IEEE CS Press, 570–584.

[28] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (2011), 117–128.

[29] Yicheng Jin, Yongji Wu, Wenjun Hu, Bruce M. Maggs, Xiao Zhang, and Danyang Zhuo. 2024. Curator: Efficient Indexing for Multi-Tenant Vector Databases. *CoRR* abs/2401.07119 (2024).

[30] Alistair Johnson, Lucas Bulgarelli, Tom Pollard, Brian Gow, Benjamin Moody, Steven Horng, Leo Anthony Celi, and Roger Mark. 2024. MIMIC-IV. *PhysioNet* (2024). https://doi.org/10.13026/kpb9-mt58 Version 3.1.

[31] Alistair Johnson, Lucas Bulgarelli, Lu Shen, Alvin Gayles, Ayad Shammout, Steven Horng, Tom Pollard, Sicheng Hao, Benjamin Moody, Brian Gow, Li-wei Lehman, Leo Celi, and Roger Mark. 2023. MIMIC-IV, a freely accessible electronic health record dataset. *Sci. Data* 10, 1 (2023), 1.

[32] Georgios Kellaris, George Kollios, Kobbi Nissim, and Adam O'Neill. 2016. Generic Attacks on Secure Outsourced Databases. In *Proc. CCS'16*. ACM, 1329–1340.

[33] Jingyu Li, Zhicong Huang, Min Zhang, Cheng Hong, Jian Liu, Tao Wei, and Wenguang Chen. 2025. Panther: Private Approximate Nearest Neighbor Search in the Single Server Setting. In *Proc. CCS'25*. ACM, 365–379.

[34] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.

[35] Frank McSherry and Kunal Talwar. 2007. Mechanism Design via Differential Privacy. In *Proc. FOCS'07*. IEEE Computer Society, 94–103.

[36] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Umar Farooq Minhas, Jeffrey Pound, Cédric Renggli, Nima Reyhani, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2024. Incremental IVF Index Maintenance for Streaming Vector Search. *CoRR* abs/2411.00970 (2024).

[37] John X. Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M. Rush. 2023. Text Embeddings Reveal (Almost) As Much As Text. In *Proc. EMNLP'23*. Association for Computational Linguistics, 12448–12460.

[38] Muhammad Naveed, Seny Kamara, and Charles V. Wright. 2015. Inference Attacks on Property-Preserving Encrypted Databases. In *Proc. CCS'15*. ACM, 644–655.

[39] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3 (2024), 120.

[40] Zipeng Qiu, Wenjie Qu, Jiaheng Zhang, and Binhang Yuan. 2026. V3DB: Audit-on-Demand Zero-Knowledge Proofs for Verifiable Vector Search over Committed Snapshots. *CoRR* abs/2603.03065 (2026).

[41] Joel Reardon, David A. Basin, and Srdjan Capkun. 2013. SoK: Secure Data Deletion. In *Proc. IEEE S&P'13*. IEEE Computer Society, 301–315.

[42] Shariq Rizvi, Alberto O. Mendelzon, S. Sudarshan, and Prasan Roy. 2004. Extending Query Rewriting Techniques for Fine-Grained Access Control. In *Proc. SIGMOD'04*, Gerhard Weikum, Arnd Christian König, and Stefan Deßloch (Eds.). ACM, 551–562.

[43] Hayim Shaul, Dan Feldman, and Daniela Rus. 2020. Secure k-ish Nearest Neighbors Classifier. *Proc. Priv. Enhancing Technol.* 2020, 3 (2020), 42–61.

[44] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *CoRR* abs/2105.09613 (2021).

[45] Sonish Sivarajkumar, Haneef Ahamed Mohammad, David Oniani, Kirk Roberts, William R. Hersh, Hongfang Liu, Daqing He, Shyam Visweswaran, and Yanshan Wang. 2024. Clinical Information Retrieval: A Literature Review. *J. Heal. Informatics Res.* 8, 2 (2024), 313–352.

[46] Congzheng Song and Ananth Raghunathan. 2020. Information Leakage in Embedding Models. In *Proc. CCS'20*. ACM, 377–390.

[47] Suhas Jayaram Subramanya, Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances of NeurIPS'19*. 13748–13758.

[48] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proc. SIGMOD'21*. ACM, 2614–2627.

[49] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2024. C-Pack: Packed Resources For General Chinese Embeddings. In *Proc. SIGIR'24*. ACM, 641–649.

[50] Wentao Xiao, Yueyang Zhan, Rui Xi, Mengshu Hou, and Jianming Liao. 2024. Enhancing HNSW Index for Real-Time Updates: Addressing Unreachable Points and Performance Degradation. *CoRR* abs/2407.07871 (2024).

[51] Haike Xu, Magdalen Dobson Manohar, Philip A. Bernstein, Badrish Chandramouli, Richard Wen, and Harsha Vardhan Simhadri. 2025. In-Place Updates of a Graph Index for Streaming Approximate Nearest Neighbor Search. *CoRR* abs/2502.13826 (2025).

[52] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proc. SOSP'23*. ACM, 545–561.

[53] Zhaozhuo Xu, Weijie Zhao, Shulong Tan, Zhixin Zhou, and Ping Li. 2022. Proximity Graph Maintenance for Fast Online Nearest Neighbor Search. *CoRR* abs/2206.10839 (2022).

[54] Tomohiro Yamashita, Daichi Amagata, and Yusuke Matsui. 2025. How Should We Evaluate Data Deletion in Graph-Based ANN Indexes? *CoRR* abs/2512.06200 (2025).

[55] Song Yu, Shengyuan Lin, Shufeng Gong, Yongqing Xie, Ruicheng Liu, Yijie Zhou, Ji Sun, Yanfeng Zhang, Guoliang Li, and Ge Yu. 2025. A Topology-Aware Localized Update Strategy for Graph-Based ANN Index. *Proc. VLDB Endow.* 19, 3 (2025), 495–508.

[56] Xinyi Zhang, Qichen Wang, Cheng Xu, Yun Peng, and Jianliang Xu. 2024. FedKNN: Secure Federated k-Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 1 (2024), V2mod011:1–V2mod011:26.

[57] Hongbin Zhong, Matthew Lentz, Nina Narodytska, Adriana Szekeres, and Kexin Rong. 2025. HoneyBee: Efficient Role-based Access Control for Vector Databases via Dynamic Partitioning. *CoRR* abs/2505.01538 (2025).

[58] Mingxun Zhou, Elaine Shi, and Giulia Fanti. 2025. PACMANN: Efficient Private Approximate Nearest Neighbor Search. In *Proc. ICLR'25*. OpenReview.net.

[59] Xing Zhou, Dmitrii Ustiugov, Haoxin Shang, and Kisson Lin. 2026. MemTrust: A Zero-Trust Architecture for Unified AI Memory System. *CoRR* abs/2601.07004 (2026).

[60] Zeqi Zhu, Zeheng Fan, Yuxiang Zeng, Yexuan Shi, Yi Xu, Mengmeng Zhou, and Jin Dong. 2024. FedSQ: A Secure System for Federated Vector Similarity Queries. *Proc. VLDB Endow.* 17, 12 (2024), 4441–4444.

Appendix Contents

A EXTENDED FORMALISM AND PROOFS

This appendix consolidates the formal model of §2–§6, completes the proofs the body defers, and gives the complexity analysis behind its asymptotic claims. All notation is that of the body; Table 6 collects it in one place. We fix the model (§A.1), restate the six axes as formal predicates (§A.2), give the full impossibility argument (§A.3), prove that Lethe achieves per-view erasure (§A.4), analyze its cost (§A.5), and state the assumptions and scope of the guarantees (§A.6).

A.1 Notation and model

A.1.1 Index and graph. A graph vector index is a pair $\mathcal{I} = (G, S)$. The *navigable graph* $G = (P, E)$ is a directed graph over a dataset $P = \{p_1, \ldots, p_N\} \subset \mathbb{R}^d$ of N vectors in d dimensions; each node p stores an adjacency list of out-neighbors $N_{\text{out}}(p)$ capped at an out-degree bound M, and we write $N_{\text{in}}(p) = \{p' \in P : p \in N_{\text{out}}(p')\}$ for its in-neighbors. The *physical store* S holds the bytes of every $p \in P$. The graph is *navigable* when, from a fixed entry point ep, greedy descent reaches the neighborhood of a query in a few hops, which requires that the nodes along the way carry edges toward it [34, 47].

A.1.2 Search semantics. A query q for principal u is answered by greedy best-first traversal (Alg. 2). The search keeps a frontier W of the ef candidates nearest q seen so far; it repeatedly expands the closest unexpanded candidate by visiting its out-neighbors, and stops when W no longer improves. We write $\text{Visit}(\mathcal{I}, q, u) \subseteq P$ for the set of nodes *expanded* during this traversal and $\text{Ans}(\mathcal{I}, q, u)$ for the k visited nodes nearest q that are returned; both are the principal-aware forms of $\text{Visit}(q, G)$ and $R_k(q, G)$ from §2. Quality is measured against the exact neighbors $\text{NN}_k(q)$ by $\text{Recall@}k = |\text{Ans}(\mathcal{I}, q, u) \cap \text{NN}_k(q) \cap P_u|/k$, the fraction of the true k nearest *authorized* neighbors recovered.

For a fixed run on (q, u) we attach to each node p six monotone event predicates that record how far p participates in the traversal; they make the informal stages of §2 and the leakage instrumentation of §7 precise.

- $\text{enc}(p)$: p's identifier is *encountered*, i.e. it appears in an adjacency list the search scans.
- $\text{chk}(p)$: the access predicate $u \in A(p)$ is *evaluated* for p.
- $\text{scr}(p)$: p is *distance-scored*, i.e. its distance to q is computed.
- $\text{ins}(p)$: p is *inserted* into the frontier W.
- $\text{exp}(p)$: p is *expanded*; equivalently $p \in \text{Visit}(\mathcal{I}, q, u)$.
- $\text{ret}(p)$: p is *returned*; equivalently $p \in \text{Ans}(\mathcal{I}, q, u)$.

These hold in the order $\text{ret}(p) \Rightarrow \text{exp}(p) \Rightarrow \text{ins}(p) \Rightarrow \text{scr}(p) \Rightarrow \text{chk}(p) \Rightarrow \text{enc}(p)$: a node cannot be returned without having been expanded, expanded without having entered the frontier, and so on. We call p a *routing waypoint* for (q, u) when $\text{exp}(p)$ holds, and we say p *participates* in the traversal when $\text{scr}(p) \vee \text{ins}(p) \vee \text{exp}(p) \vee \text{ret}(p)$ holds. As the body's footnote to Table 1 notes, $\text{enc}(p)$ alone is benign: p may sit in the adjacency list of a node still shared with views that hold it, so its identifier can be encountered without p participating.

A.1.3 Access-control model. A shared index serves a set of principals U (users or roles). An access policy assigns each vector p an authorized set $A(p) \subseteq U$; principal u's *view* is the sub-dataset

$P_u = \{p \in P : u \in A(p)\}$ it may retrieve. Permissions form a lattice $(L, \sqsubseteq)$ of roles, where $\ell' \sqsubseteq \ell$ means role ℓ subsumes (*dominates*) the access of ℓ'. Each principal u holds a role $\lambda(u) \in L$ and each vector p is tagged with the least role $\lambda(p) \in L$ permitted to see it, so u may retrieve p exactly when $\lambda(p) \sqsubseteq \lambda(u)$, i.e. $A(p) = \{u \in U : \lambda(p) \sqsubseteq \lambda(u)\}$. For a role $\ell \in L$ we write $P_\ell = \{p \in P : \lambda(p) \sqsubseteq \ell\}$ for the vectors visible at ℓ; a principal's view is the view of its role, $P_u = P_{\lambda(u)}$, so principals sharing a role share a view.

A principal's *effective view* is P_u once individual exceptions are applied. Principals with the same effective view form an *effective-view class*; we write $\mathrm{ev}(u)$ for the class of u, which is its role $\lambda(u)$ when no individual exception applies and a refinement of it otherwise. The access test is realized by storing, with each p, its role tag $\lambda(p)$ and a short *exception list* of principals revoked from the role-induced default; $u \in A(p)$ then checks $\lambda(p) \sqsubseteq \lambda(u)$ against a precomputed reachability relation on L and consults the exception list, in $O(1)$ (§6.3, formalized in §A.5).

A.1.4 Erasure requests and the reference view. A *revocation* $\mathrm{revoke}(u, p)$ removes u from $A(p)$, shrinking P_u by one vector and leaving $P_{u'}$ unchanged for every other principal. An *erasure request* $\rho = (p, R)$ names a vector p and a scope $R \subseteq U$ of principals who must lose access to it: $R = \{u\}$ is a single revocation and $R = U$ is *global erasure*. Applying ρ transforms $\mathcal{I} = (G, S)$ into $\mathcal{I}' = (G', S')$ with $u \notin A(p)$, hence $p \notin P_u$, for every $u \in R$. We write $\mathcal{L}(R)$ for the set of effective-view classes whose view loses p under ρ: one class for a single-role revocation, the class $\mathrm{ev}(u)$ for $R = \{u\}$, and every class whose view contained p for $R = U$.

To say what u *should* observe once p leaves its view, we fix a reference $\mathcal{I}_u^\star$, the index built on exactly P_u – the oracle per-view rebuild of §3, the behavior an index would have if p, and every other vector u cannot see, had never been inserted for u. The reference is an empirical quality target, not part of the guarantee: Definition 4.1 pins down p's operational non-participation exactly, while §7 shows the resulting answers track $\mathcal{I}_u^\star$ to within its own seed-to-seed variation.

A.2 The six axes, formally

Table 1 states each axis as a condition on the post-request index $\mathcal{I}'$. We restate the six as formal predicates over the event predicates of §A.1.2, so that the impossibility argument (§A.3) and the correctness proof (§A.4) reduce to checking them. Fix a request $\rho = (p, R)$ taking $\mathcal{I} = (G, S)$ to $\mathcal{I}' = (G', S')$.

A.2.1 The axis predicates.

Results. For every $u \in R$ and every query q, $p \notin \mathrm{Ans}(\mathcal{I}', q, u)$; equivalently $\neg\mathrm{ret}(p)$ on every run (q, u) with $u \in R$.

Routing. For every $u \in R$ and every query q, p does not *participate*: $\neg\big(\mathrm{scr}(p) \vee \mathrm{ins}(p) \vee \mathrm{exp}(p) \vee \mathrm{ret}(p)\big)$. In particular $p \notin \mathrm{Visit}(\mathcal{I}', q, u)$. The benign event $\mathrm{enc}(p)$ is permitted, since p may remain a physical node shared with views that still hold it (footnote to Table 1).

Physical. If $R = U$, then p's bytes are unrecoverable from S': there is no efficient procedure that, given read access to live memory and to every serialized snapshot of S', outputs p's plaintext (formalized under stated assumptions in §A.4.5).

Immediate. The Results and Routing predicates hold for every query accepted at or after the time ρ is accepted; there is a single instant (the linearization point of §A.4.4) before which ρ has no effect and after which both hold.

Verifiable. There is a tamper-evident record $\tau(\rho)$ such that a third party holding the public verification state can confirm ρ was issued and ordered, and any alteration of the recorded history is detected (formalized in §A.4.6).

View-scope. For every $u' \notin R$ with $p \in P_{u'}$, the run (q, u') is unchanged in its answer and in p's participation: $p \in \mathrm{Ans}(\mathcal{I}', q, u') \iff p \in \mathrm{Ans}(\mathcal{I}, q, u')$, and likewise for $\mathrm{exp}(p)$; and u' keeps its recall up to the seed-level drift measured in §7.

Definition A.1 (Per-view erasure; restatement of Definition 4.1). A mechanism achieves *per-view erasure* if, for every request $\rho = (p, R)$, the index $\mathcal{I}'$ it produces satisfies Results, Routing, Immediate, Verifiable, and View-scope for scope R, and additionally Physical when $R = U$.

A.2.2 Routing is strictly stronger than Results. The body keeps Results and Routing separate because they are not equivalent; we make the implication and its failure precise.

Lemma A.2 (Routing implies Results). *For any index and any scope R, if $\mathcal{I}'$ satisfies Routing for $\rho = (p, R)$ then it satisfies Results for ρ.*

Proof. By the ordering of §A.1.2, $\mathrm{ret}(p) \Rightarrow \mathrm{exp}(p)$, so $\mathrm{ret}(p)$ implies p participates. Routing forbids participation for every $u \in R$ and every q, hence forbids $\mathrm{ret}(p)$, which is exactly Results. □

Lemma A.3 (Results does not imply Routing). *There is an index, a request $\rho = (p, R)$, and a query q for which $\mathcal{I}'$ satisfies Results but violates Routing.*

Proof. Take the running example of Fig. 2, in which the bridge b lies on the greedy path to q – $\mathrm{exp}(b)$ holds – yet $b \notin R_k(q, G)$, so $\mathrm{ret}(b)$ does not. Let $\rho = (b, \{u\})$ and let $\mathcal{I}'$ be produced by a mechanism that filters b from the answer but leaves it a node of G' with its edges intact (the Tombstone behavior of §3). Then $\neg\mathrm{ret}(b)$ on (q, u), so Results holds, while $\mathrm{exp}(b)$ still holds, so b participates and Routing fails. The participation of b moves the frontier and can change which other vectors are returned: this is the zombie channel whose drift §3 measures. □

Lemmas A.2–A.3 place Routing strictly above Results in the order of guarantees: an access check applied only at the output enforces the latter but never the former.

A.2.3 Per-view leakage. Definition A.1 drives to zero the extent to which p still reaches a revoked view. We name that quantity. For a request $\rho = (p, R)$, a revoked principal $u \in R$, and a distribution Q over queries, define the *per-view leakage*

$$\mathrm{Leak}(p, u) = \Pr_{q \sim Q}\big[\,\mathrm{scr}(p) \vee \mathrm{ins}(p) \vee \mathrm{exp}(p) \vee \mathrm{ret}(p) \text{ on run } (q, u)\,\big],$$

the probability that p participates in the traversal of a query issued by u after revocation. Summing the disjuncts by stage gives the stage-resolved leakage, scored, frontier, expanded, and returned (result) leakage, that §7 reports and the instrumentation of §7 records

Table 6: Symbols used in the body and this appendix, with the section of first use. Notation is identical throughout; the event predicates of §A.1.2 make the leakage stages of §7 precise.

Symbol	Meaning	First use
$\mathcal{I} = (G, S)$	Index: navigable graph G and physical store S	§2, §4
$P = \{p_1, \ldots, p_N\}$	Dataset of N vectors in $\mathbb{R}^d$	§2
$G = (P, E)$	Directed navigable graph over P	§2
$N_{\text{out}}(p),\ N_{\text{in}}(p)$	Out- and in-neighbors of node p	§2
M	Out-degree bound	§2
$ep,\ ef$	Entry point; frontier width	§2, §6
W	Frontier of authorized candidates during search	§6
$\text{Visit}(\mathcal{I}, q, u)$	Nodes expanded for query q, principal u	§4
$\text{Ans}(\mathcal{I}, q, u)$	Top-k returned over u's view	§4
$R_k(q, G)$	k returned neighbors (view-agnostic form)	§2
$\text{NN}_k(q)$	Exact k nearest neighbors of q	§2
Recall@k	Fraction of authorized true neighbors recovered	§2
enc, chk, scr	Encountered / predicate-checked / distance-scored	§A.1.2
ins, exp, ret	Frontier-inserted / expanded / returned	§A.1.2
U	Set of principals (users or roles)	§2
$A(p)$	Authorized set of p (access predicate)	§2, §5
$P_u,\ P_\ell$	View of principal u; vectors visible at role ℓ	§2
$(L, \sqsubseteq)$	Permission lattice of roles	§2
$\lambda(u),\ \lambda(p)$	Role of principal u; least role tagging p	§2
$\text{ev}(u)$	Effective-view class of u	§6
$\text{revoke}(u, p)$	Revocation: remove u from $A(p)$	§2
$\rho = (p, R)$	Erasure request: vector p, scope $R \subseteq U$	§4
$\mathcal{L}(R)$	Effective-view classes whose view loses p	§6
$\mathcal{I}_u^\star$	Oracle reference index built on exactly P_u	§4
$\text{Bypass}_\ell(a)$	Bypass edges of a tagged class ℓ	§6
$\text{Prune}(x, cand, M)$	Robust neighbor-selection heuristic	§5

per query; *operational leakage* is the expanded component, the rate at which a revoked vector is still a routing waypoint.

Proposition A.4. *A mechanism satisfies* Results *and* Routing *for $\rho = (p, R)$ if and only if* $\text{Leak}(p, u) = 0$ *for every $u \in R$ and every query distribution $\mathcal{Q}$.*

Proof. Routing forbids each disjunct in the definition of Leak for every $u \in R$ and every q, so the probability is 0 under any $\mathcal{Q}$; with Lemma A.2 this also gives Results. Conversely, if $\text{Leak}(p, u) = 0$ for every $\mathcal{Q}$ then, choosing $\mathcal{Q}$ to be the point mass at an arbitrary q, no disjunct holds on (q, u); as this is exactly the participation of p, Routing holds, and Results follows. □

This is a structural property of the index: it bounds p's operational participation, and is separate from inference about hidden data from public structure, which we do not study (§A.6).

A.2.4 The axes are independent. The body calls the six axes independent. We make this precise: no axis is implied by the conjunction of the others, so none is redundant.

Proposition A.5. *For each of the six axes there is a mechanism that satisfies the other five (on the requests where they apply) but violates it. Hence the six are logically independent, and Definition A.1 is not implied by any five of its conjuncts.*

Proof. We exhibit a witness per axis; the per-primitive analysis of §A.3 and the matrix of Fig. 1 supply the behaviors invoked.

Routing. A mechanism that filters p from answers, audits the revocation, scopes it to R, and crypto-shreds at $R = U$, but leaves p routing, satisfies all axes except Routing (Lemma A.3).

Results. Returning p while otherwise erasing it is precluded by Lemma A.2 only when Routing holds; a mechanism that suppresses participation in scoring, expansion, and the frontier but admits p into the final answer set by a separate output path violates Results alone. Such a mechanism is degenerate but well defined, witnessing that Results is not implied by the rest.

Physical. Lethe without cryptographic shredding (§7, ablation) satisfies every axis but, at $R = U$, leaves p's bytes in the live store, violating Physical.

Immediate. A mechanism that defers the predicate flip and bypass to a periodic pass attains all axes once it runs but violates Immediate until then (the Periodic-rebuild behavior).

Verifiable. Lethe without the erasure log satisfies every operational axis but produces no tamper-evident record, violating Verifiable.

View-scope. A global physical deletion attains Results, Routing, Physical, and Immediate but removes p for every principal, violating View-scope.

Each axis thus has a witness satisfying the other five, so none follows from their conjunction. □

Proposition A.5 is the formal content behind the empty and filled cells of Fig. 1: each prior method occupies a distinct pattern of satisfied axes, and only their conjunction, which no single mechanism in the figure attains, is per-view erasure.

A.3 No standard primitive achieves per-view erasure

Theorem 4.2 states that none of the standard maintenance primitives, applied alone, achieves per-view erasure. The body gives the argument in prose; here we prove one lemma per primitive against the formal predicates of §A.2.1, then draw the two structural conclusions that motivate Lethe (§A.3.2). Throughout, $\rho = (p, R)$ is an erasure request and $\mathcal{I}'$ the index the primitive produces; Fig. 1 records the full pattern of satisfied axes.

A.3.1 Per-primitive lemmas.

Lemma A.6 (Tombstone). *Tombstone satisfies Results and Immediate but violates Routing, Physical, Verifiable, and View-scope.*

Proof. Tombstone sets a deleted flag on p and filters flagged nodes from the returned set, so $\neg\mathrm{ret}(p)$ on every run: Results holds, and because the flag is read live it holds from the next query (Immediate). But p remains a node of G' with $N_{\mathrm{out}}(p)$ and $N_{\mathrm{in}}(p)$ intact; nothing in the traversal consults a per-view predicate, so p is still distance-scored, inserted, and expanded on a revoked user's query. Thus $\exp(p)$ holds, p participates, and Routing fails (the drift this induces is measured in §3). The bytes of p stay in the live store and in every snapshot, so for $R = U$ they remain recoverable: Physical fails. A mutable flag is not a tamper-evident record: Verifiable fails. And the flag is global, it suppresses p for every principal at once, with no dependence on R, so a request $R = \{u\}$ also denies p to principals $u' \neq u$ that should keep it: View-scope fails. □

Lemma A.7 (Physical-delete). *Physical-delete satisfies Results, Routing, Physical, and Immediate but violates Verifiable and View-scope.*

Proof. Physical-delete removes p's node from G' and repairs its in-neighbors $N_{\mathrm{in}}(p)$ by re-pointing them to suitable survivors. After it runs, p is no node of G', so no event predicate beyond enc can fire: p is neither scored, expanded, nor returned, giving Routing and Results; its bytes are gone from the live store, giving Physical for $R = U$; and the in-place edit takes effect at once (Immediate). But the deletion acts on the single shared graph, hence on every principal regardless of R: a request $R = \{u\}$ removes p from $P_{u'}$ for $u' \neq u$ that still hold it, so View-scope fails. And the edit leaves no auditable artifact attesting the request occurred: Verifiable fails. □

Lemma A.8 (Periodic-rebuild). *Periodic-rebuild satisfies Results and Routing once it runs but violates Physical, Immediate, Verifiable, and View-scope.*

Proof. At the next scheduled pass, Periodic-rebuild reconstructs the graph on the surviving set, after which p is absent and neither routes nor appears in any answer (Routing, Results). But the pass is deferred: between acceptance of ρ and the next rebuild, p stays a node of the live graph and store, so it keeps routing and its bytes stay recoverable. Hence Immediate fails, and for $R = U$ Physical fails during the entire compliance window of §3. The rebuild is over the single shared set, so it is global (View-scope fails), and it produces no tamper-evident record of the request (Verifiable fails). □

Lemma A.9 (Cryptographic shredding). *Cryptographic shredding satisfies Physical, Immediate, and Verifiable but violates Results, Routing, and View-scope.*

Proof. Shredding destroys p's key, so its ciphertext in S' becomes undecryptable: the plaintext is unrecoverable (Physical), the destruction is effective at once (Immediate), and a key-destruction event can be logged checkably (Verifiable). But shredding changes the bytes, not the graph: p's node and edges remain in G' and no predicate excludes them, so the search still scores, expands, and can return p. Thus $\exp(p)$ and possibly $\mathrm{ret}(p)$ hold, so Routing and Results fail; the distance p then contributes is computed on unreadable bytes and is meaningless rather than absent, which corrupts rather than removes its routing influence. Finally a key binds to a record, not to a view, so destroying it affects every principal: View-scope fails (and the act is defined only for $R = U$). □

A.3.2 Two structural conclusions.

Proof of Theorem 4.2. Each of Lemmas A.6–A.9 exhibits an axis its primitive violates, so none satisfies all of Definition A.1's conjuncts; hence none, applied alone, achieves per-view erasure. □

Corollary A.10. *No standard primitive satisfies View-scope, and none satisfies Routing, Physical, Immediate, and View-scope together.*

Proof. Lemmas A.6–A.9 each list View-scope among the violated axes, because each primitive acts on the single shared structure (graph, store, or key) and so cannot confine an erasure to a chosen scope $R \subsetneq U$. The View-scope column of Fig. 1 is therefore empty, which already rules out the joint conjunction in the claim. □

The proof points to what a solution must combine.

Lemma A.11 (Necessary combination). *Any mechanism achieving per-view erasure must provide (i) view-scoped control of both Results and Routing, denying p to the revoked views in R while leaving every $u' \notin R$ with $p \in P_{u'}$ undisturbed; (ii) byte destruction of p when $R = U$; and (iii) a single operation that is immediate and leaves a tamper-evident record.*

Proof. Routing and Results for $u \in R$ require suppressing p's participation on revoked views, while View-scope forbids changing Ans or $\exp(p)$ for $u' \notin R$; together these demand control indexed by view, not by record, which is (i). Physical at $R = U$ requires p's bytes be unrecoverable from S', which only byte destruction provides, giving (ii). Immediate requires the operational predicates to hold from a single acceptance instant, and Verifiable requires

a tamper-evident artifact of the request; an unaudited or deferred mechanism fails one or the other, giving (iii). No single primitive of §A.3.1 supplies all three (Corollary A.10), so a mechanism achieving per-view erasure must combine them, as Lethe does (§5, proved in §A.4). □

A.3.3 Filtered search and per-view materialization. Two further classes of method appear as baselines in §7; we place them against the axes for completeness. They are not counterexamples to Theorem 4.2, one shares Tombstone's failure mode, the other attains the axes but at a cost the rest of this appendix quantifies, but a hostile reviewer will ask where they fall.

Lemma A.12 (Post-filter). *Post-filtering, which runs the unmodified search and removes unauthorized vectors from the returned set, satisfies Results and Immediate but violates Routing, Physical, Verifiable, and View-scope.*

Proof. Removing p from the output after the search gives $\neg\mathsf{ret}(p)$ (Results) and applies on the next query (Immediate). But the search itself is unmodified: p is scored and expanded exactly as before, so $\mathsf{exp}(p)$ holds and Routing fails, with the same zombie effect as Tombstone. The vector's bytes and node are untouched (Physical fails for $R = U$), there is no audit artifact (Verifiable fails), and filtering is applied uniformly without a per-view scope on the index (View-scope fails). Post-filter thus shares Tombstone's pattern on the operational axes. □

The remaining baselines, the per-role and per-user materialized indexes, *do* achieve per-view erasure: each view owns a private index, so a revocation rebuilds only that view and never touches another, satisfying every axis including View-scope. They fail not on correctness but on feasibility.

Proposition A.13 (Per-view materialization is correct but space-prohibitive). *A per-role materialization stores one graph per role and so consumes $\Theta(|L| \cdot N \cdot M)$ edge slots in the worst case; a per-user materialization consumes $\Theta(|U| \cdot N \cdot M)$. Both achieve per-view erasure, but their space grows with the number of distinct views rather than with the number of revocations.*

Proof. Materializing a separate navigable graph for each of the $|L|$ roles stores up to N nodes with out-degree M per role, i.e. $\Theta(|L| \cdot N \cdot M)$ edges; the per-user case replaces $|L|$ by $|U|$. Because each view's index is independent, a request $\rho = (p, R)$ edits only the indexes of the views in R, leaving all others bit-identical, which gives View-scope and, together with an in-place delete-and-repair per affected index, the remaining axes. The cost is therefore set by the number of materialized views, not by the request, and is the baseline that §A.5 compares against Lethe's overlay, whose size scales with revocations and shared effective views instead. □

Proposition A.13 frames the design problem the rest of the paper solves: attain the per-view guarantee of a materialized index at a cost that scales with what is revoked, not with how many views exist. The memory measurements of §7 and the bounds of §A.5 quantify the gap.

A.4 Correctness of Lethe

We prove that Lethe achieves per-view erasure (Definition A.1). Each axis is established as a lemma about the algorithms of §6: revocation-aware search (Alg. 2), bypass construction (Alg. 1), and maintenance (Alg. 3). The lemmas assemble into the main theorem (§A.4.7); crash and concurrency behavior follow (§A.4.8). Throughout, $\rho = (p, R)$ is the request, $\mathcal{I}' = (G', S')$ the resulting index, and $\ell = \mathrm{ev}(u)$ the effective-view class of a principal u; $A(p)$ denotes the live access predicate after maintenance, i.e. with R already removed.

A.4.1 Transparency invariant: Results and Routing. A node b is *transparent* to u when $u \notin A(b)$. The search admits a node into the frontier only after the predicate test passes; we show this suffices to suppress every form of participation.

Lemma A.14 (Transparency invariant). *Fix a query q and principal u, and run Alg. 2 on $\mathcal{I}'$. Every node b with $u \notin A(b)$ satisfies, on this run, $\neg\mathsf{ins}(b)$, $\neg\mathsf{exp}(b)$, and $\neg\mathsf{ret}(b)$, and its distance to q never determines which node is expanded next. In particular, for the revoked vector p and every $u \in R$, p does not participate: Results and Routing hold.*

Proof. Alg. 2 maintains the frontier W and inserts a freshly visited node b only inside the guarded branch $u \in A(b)$ (the line "**if** $u \in A(b)$**:** $W \leftarrow W \cup \{b\}$"). Hence if $u \notin A(b)$, b is never added to W: $\neg\mathsf{ins}(b)$. The loop expands and returns only nodes drawn from W, it picks the nearest *unexpanded node of* W to expand and returns the k nearest nodes *of* W, so a node never in W is never expanded or returned: $\neg\mathsf{exp}(b)$ and $\neg\mathsf{ret}(b)$. The choice of which node to expand and which to return depends only on distances of nodes in W; since $b \notin W$, its distance enters neither comparison, so it cannot steer the search. Applying this to p, which has $u \notin A(p)$ for every $u \in R$ after maintenance, gives $\neg\mathsf{ret}(p)$ (Results) and $\neg(\mathsf{ins}(p) \vee \mathsf{exp}(p) \vee \mathsf{ret}(p))$ together with no routing influence; p is distance-scored at most to evaluate the guard, which by §A.1.2 is the benign chk, not participation. Hence p does not participate and Routing holds. □

A.4.2 Bypass navigability: recall under View-scope. Making p transparent removes a connector from u's subgraph; the bypass overlay restores the reachability a rebuild on P_ℓ would have provided. We state what it restores and the sense in which recall is preserved.

Lemma A.15 (Bypass navigability). *Let p be transparent across effective-view class ℓ, and let* Bypass_ℓ *be the edges Alg. 1 adds. For every authorized in-neighbor $a \in N_{\mathrm{in}}(p) \cap P_\ell$ and every authorized survivor $c \in C = N_{\mathrm{out}}(p) \cap P_\ell$ that a reached through p, the augmented graph $G_\ell = (P_\ell,\ (E \restriction P_\ell) \cup \mathrm{Bypass}_\ell)$ contains a path from a to c that does not pass through p, with every edge having both endpoints in P_ℓ and out-degree bounded by M. Consequently a greedy search restricted to P_ℓ over G_ℓ explores the region a rebuild on P_ℓ would, up to the approximation of* Prune.

Proof. Alg. 1 sets $C = N_{\mathrm{out}}(p) \cap P_\ell$ and, for each $a \in N_{\mathrm{in}}(p) \cap P_\ell$, computes $T = \mathrm{Prune}\big(a,\ (N_{\mathrm{out}}(a) \cup C) \cap P_\ell,\ M\big)$ and adds the edges $a \to b$ for $b \in T \setminus N_{\mathrm{out}}(a)$, tagged ℓ. The candidate set passed to Prune contains C, and Prune returns close, diverse neighbors capped at M; by the same selection property that makes the base

graph navigable (§2), each survivor $c \in C$ is either retained directly in T or reachable from a retained neighbor, so a reaches c along edges in $E \restriction P_\ell \cup \text{Bypass}_\ell$ without traversing p. Each added edge is selected from P_ℓ, so both endpoints lie in P_ℓ, and $|T| \le M$ bounds the out-degree like the base graph. This is the same local repair a physical deletion of p confined to P_ℓ would apply (§5), so greedy descent over G_ℓ reaches the neighborhood of a query as on a rebuilt index, differing only where Prune's heuristic selection differs from a full reconstruction. □

Remark A.16. *Lemma A.15 gives reachability, not edge-for-edge identity with a rebuild: the overlay restores navigability up to the pruning heuristic, so the View-scope recall guarantee is the empirical one of Table 1, authorized recall tracks the oracle per-view rebuild to within seed-level drift, measured in §7, rather than a claim of identical answers.*

A.4.3 View-scope isolation. A request must not disturb a principal who keeps p. We show maintenance changes nothing observable for such a principal.

Lemma A.17 (View-scope isolation). *Let $\rho = (p, R)$ be applied by Alg. 3, and let $u' \notin R$ with $p \in P_{u'}$. For every query q, the run (q, u') on $\mathcal{I}'$ visits, scores, expands, and returns exactly the nodes it did on $\mathcal{I}$; in particular $\text{Ans}(\mathcal{I}', q, u') = \text{Ans}(\mathcal{I}, q, u')$ and $\exp(p)$ is unchanged. View-scope holds.*

Proof. Alg. 3 makes three kinds of change. (i) It updates the predicate to $A(p) \setminus R$; since $u' \notin R$, the test $u' \in A(p)$ is unchanged, so p remains non-transparent to u' and Alg. 2 treats it exactly as before. (ii) It adds bypass edges tagged with classes $\ell \in \mathcal{L}(R)$. By construction u' keeps p, so $\text{ev}(u') \notin \mathcal{L}(R)$, and Alg. 2 follows only $\text{Bypass}_{\text{ev}(u')}$; edges tagged $\ell \neq \text{ev}(u')$ are never traversed on u''s queries. (iii) For $R = U$ it destroys p's key; but $R = U$ means $u' \in R$, contradicting $u' \notin R$, so this branch does not execute for any u' in scope of the claim. No change therefore alters u''s traversal, and its visited, scored, expanded, and returned sets, hence its answer and $\exp(p)$, are identical on $\mathcal{I}'$ and $\mathcal{I}$. □

A.4.4 Immediacy and the linearization point. Immediate requires a single instant after which the operational guarantees hold. We identify it with the predicate commit and show the remaining work may lag.

Lemma A.18 (Immediacy). *Define the* linearization point *of $\rho = (p, R)$ as the commit of the predicate update $A(p) \leftarrow A(p) \setminus R$ in Alg. 3. Every query accepted after this instant satisfies Results and Routing for p on every revoked view, independently of whether the bypass build (Alg. 1) and the log append have completed.*

Proof. Once the predicate commit is published, every subsequent run of Alg. 2 for $u \in R$ reads $u \notin A(p)$ and, by Lemma A.14, p does not participate: Results and Routing hold from that query onward, which is Immediate. The bypass edges and the log entry bear on recall and audit, not on whether p is denied: a query before the bypass completes still excludes p correctly and merely follows base edges, losing at most the recall the overlay would restore (Lemma A.15); the log append records the event but does not gate the denial. Hence neither lagging step affects Results or Routing. □

A.4.5 Physical unrecoverability. For a global request, Physical requires p's bytes be unrecoverable. Lethe reduces this to key destruction; we state the assumptions the reduction needs and prove it under them.

Assumption A.19. *The store S encrypts each vector under a per-vector key, with (a)* key independence*: p's key is information-theoretically independent of every other vector's key and of the ciphertexts; (b)* trusted destruction*: key destruction in the key store erases every copy of p's key from memory and persistent media, e.g. inside a trusted execution environment; and (c)* semantic security*: the cipher is IND-CPA secure, so ciphertext without the key reveals nothing about the plaintext beyond its length.*

Lemma A.20 (Physical unrecoverability). *Under Assumption A.19, after Alg. 3 processes a global request $\rho = (p, U)$, no efficient procedure with read access to live memory and to every serialized snapshot of S' outputs p's plaintext with non-negligible advantage. Physical holds.*

Proof. For $R = U$ the algorithm destroys p's key; by (b) no copy of the key survives in memory or on disk. What remains accessible is p's ciphertext in S' and state derived without p's key. By (a) no other key functions to decrypt p, and (c) makes the ciphertext indistinguishable from the encryption of an unrelated message of equal length: any procedure recovering the plaintext with non-negligible advantage would break IND-CPA. Live memory and snapshots contain only such ciphertext and key-independent metadata (the graph, the predicate bitmaps, the overlay, the log), none of which carries p's plaintext. Hence the plaintext is unrecoverable, which is Physical. The assumptions are necessary: dropping (b) leaves a retained key that trivially decrypts, dropping (a) lets a related key do so, and dropping (c) admits ciphertext-only recovery; §A.6 discusses this. □

A.4.6 Verifiability. Verifiable requires a tamper-evident record that a third party can check. The erasure log provides it.

Lemma A.21 (Tamper-evidence). *The append-only erasure log is a hash chain whose i-th record is $\tau_i = \langle p_i, R_i, t_i, \text{commit}(p_i), h_i \rangle$ with $h_i = H(\tau_{i-1} \| p_i \| R_i \| t_i \| \text{commit}(p_i))$ for a collision-resistant hash H, signed under the operator's key. A third party holding the head h_n and the public verification key can confirm that a request $\rho = (p, R)$ was recorded and ordered, and any insertion, deletion, reordering, or modification of records is detected except with negligible probability.*

Proof. To confirm ρ took effect, the verifier checks that some record τ_i carries (p, R) and the predicate commit $\text{commit}(p)$, that the chain recomputes h_j for all $j \le i$ from the genesis record, and that the signature on the head verifies. Tamper-evidence reduces to collision resistance: altering, inserting, or deleting any τ_j changes h_j, which by the chaining $h_{j+1} = H(\tau_j \| \cdots)$ propagates to every later link and to the signed head; producing a different history with the same head requires a hash collision or a signature forgery, both negligible. Reordering two records likewise changes the recomputed chain. Hence the recorded history is verifiable and tamper-evident, which is Verifiable. □

A.4.7 Main correctness theorem.

Theorem A.22 (Lethe achieves per-view erasure). *Under Assumption A.19, for every request $\rho = (p, R)$ the index Lethe produces satisfies Results, Routing, Immediate, Verifiable, and View-scope for scope R, and additionally Physical when $R = U$. Hence Lethe achieves per-view erasure in the sense of Definition A.1.*

Proof. Results and Routing hold by Lemma A.14; View-scope holds for unaffected principals by Lemma A.17, with authorized recall preserved by Lemma A.15 and Remark A.16; Immediate holds by Lemma A.18; Verifiable holds by Lemma A.21; and for $R = U$, Physical holds by Lemma A.20. These are exactly the conjuncts of Definition A.1, so Lethe achieves per-view erasure. By Lemma A.11 it does so by supplying the three capabilities no single primitive of §A.3 provides: view-scoped control of results and routing (the predicate and overlay), byte destruction at $R = U$ (shredding), and an immediate, auditable operation (the predicate commit and the log). □

A.4.8 Crash consistency and concurrency.

The linearization point also governs recovery and concurrent execution.

Lemma A.23 (Crash consistency). *If a crash interrupts Alg. 3, recovery restores the committed predicate state and, for a global request, confirms p's key is destroyed. A request whose predicate commit completed is never lost, and a vector denied before the crash never reappears in Results or Routing after recovery.*

Proof. The predicate commit is the durable point of the request: if it completed before the crash, recovery reads $A(p) \setminus R$ and, by Lemma A.14, p stays denied to every $u \in R$; if it did not, the request had not yet taken effect and is retried. The bypass and log are idempotent followers, a bypass for (p, ℓ) is built only "if not yet built", and the log append is keyed by request, so re-running maintenance after recovery reconstructs them without duplication. For $R = U$, recovery checks the key store and completes destruction if the key survived, after which Lemma A.20 applies. If recovery cannot confirm the committed state it falls back to denying p (safe mode), so a denied vector never reappears. □

Lemma A.24 (Concurrency). *Under concurrent queries and maintenance, the predicate commit provides a single linearization point per request: every query observes either the pre-request or the post-request predicate for p, and never an intermediate state in which a revoked p both is denied and still participates. No query returns or routes through a p whose revocation it linearizes after.*

Proof. A query reads the access predicate for each visited node through a single atomic lookup, so for p it observes either $A(p)$ or $A(p) \setminus R$, fixed at the lookup. If it observes $A(p) \setminus R$ with $u \in R$, then by Lemma A.14 p does not participate; if it observes the old $A(p)$, the query is linearized before the request and p participates as before, consistently. The bypass overlay is read-only during search and grows only by adding edges tagged with a class, so a concurrent build never removes an edge a query relies on; a query either sees the new edges or follows base edges, both correct by Lemma A.18. Thus every interleaving is equivalent to a serial order in which the query falls entirely before or after the request's linearization point. □

A.5 Complexity and cost

This section bounds the cost of Lethe's operations and proves the property that distinguishes it from the materialized baselines of §A.3.3: its query overhead is asymptotically free, and its revocation time and overlay space scale with *what is revoked*, the revoked vector's authorized in-neighborhood and the affected effective views, rather than with the index size N or the number of views. The bounds use the symbols of Table 6; the measured constants that instantiate them, including the memory ratios and revocation throughput, are reported in §7 and summarized against the asymptotics in Table 4. Per-method asymptotics are collected in Table 7.

A.5.1 Query time.

Proposition A.25 (Query overhead is asymptotically free). *Revocation-aware search (Alg. 2) runs in $O(ef \cdot M \cdot d)$ time, the same order as an unmodified greedy search, plus $O(ef \cdot M)$ access-predicate tests; each test is $O(1)$. Hence the access check adds no asymptotic overhead.*

Proof. Greedy search visits $O(ef)$ frontier nodes and, at each, scans up to M out-neighbors, computing a d-dimensional distance per neighbor, for $O(ef \cdot M \cdot d)$ distance work. Alg. 2 inserts the single extra step of testing $u \in A(b)$ before admitting a neighbor b to the frontier, once per examined neighbor, i.e. $O(ef \cdot M)$ tests. The predicate is a per-vector authorization bitmap indexed by effective-view class with an exception list (§A.1.3); a class lookup is a constant-time bit read and the exception check is a constant-time membership test in the amortized hashed representation, so each test is $O(1)$. Since $O(ef \cdot M) \subseteq O(ef \cdot M \cdot d)$, the test cost is dominated by the distance work and the total order is unchanged. The bypass edges followed on a revoked view are tagged with that view's class and are read exactly like base edges, adding no per-edge cost beyond an out-degree that Prune holds at M (Lemma A.15). □

A.5.2 Revocation time.

The maintenance cost is the crux. A rebuild or a full adjacency scan is $\Theta(N)$ or worse; Lethe touches only the revoked vector's authorized in-neighbors in the affected views.

Proposition A.26 (Output-sensitive revocation). *Applying a request $\rho = (p, R)$ with Alg. 3 costs*

$$O(|R|) \;+\; O\Big(\textstyle\sum_{\ell \in \mathcal{L}(R)} |N_{\text{in}}(p) \cap P_\ell| \cdot M \cdot (M + d)\Big) \;+\; O(1),$$

for the predicate update, the bypass construction over the affected views, and the log append, respectively. This is independent of N. In contrast, Periodic-rebuild costs $\Theta(N \cdot M \cdot d)$ to reconstruct the graph and a naive in-neighbor search without stored reverse adjacency costs $\Theta(N \cdot M)$ to scan all adjacency.

Proof. Alg. 3 performs three steps. (i) It removes R from $A(p)$, touching $O(|R|)$ predicate entries. (ii) For each affected class $\ell \in \mathcal{L}(R)$ it calls Alg. 1, which iterates over the authorized in-neighbors $a \in N_{\text{in}}(p) \cap P_\ell$ and, for each, runs Prune over the candidate set $(N_{\text{out}}(a) \cup C) \cap P_\ell$ of size $O(M)$, at $O(d)$ per distance and $O(M)$ comparisons, i.e. $O(M \cdot (M + d))$ per in-neighbor; summing gives the stated term. (iii) It appends one hash-chain record, $O(1)$. None of these scans the N nodes: the in-neighbors are obtained from the stored reverse adjacency (§A.5.3), so the cost depends on

$|N_{\text{in}}(p) \cap P_\ell|$ and $|\mathcal{L}(R)|$, not on N. A rebuild instead reconstructs every node's neighborhood, $\Theta(N \cdot M \cdot d)$; locating in-neighbors by scanning adjacency lists without a reverse index visits all $\Theta(N \cdot M)$ edges. The measured revocation latency and throughput that instantiate this bound appear in §7. □

Corollary A.27. *For a fixed revocation pattern and bounded in-degree, Lethe's per-request maintenance time is asymptotically independent of the dataset size, whereas every rebuild-based or scan-based primitive grows at least linearly in N.*

Proof. Immediate from Proposition A.26: the maintenance terms involve $|R|$, $|\mathcal{L}(R)|$, $|N_{\text{in}}(p) \cap P_\ell|$, M, and d, none of which is a function of N when the in-degree is bounded; the compared primitives carry an explicit factor of N. □

A.5.3 Reverse adjacency. Output-sensitive revocation requires the in-neighbors of p to be retrievable without scanning, which a stored reverse-adjacency index provides at a bounded additive space cost.

Proposition A.28 (Reverse-adjacency cost). *Maintaining the reverse adjacency $\{N_{\text{in}}(p)\}_{p \in P}$ adds $\Theta\big(\sum_p |N_{\text{in}}(p)|\big) = \Theta(|E|) = \Theta(N \cdot M)$ entries, matching the forward graph to within a constant, and lets Alg. 3 enumerate $N_{\text{in}}(p)$ in $O(|N_{\text{in}}(p)|)$ time rather than by an $\Theta(N \cdot M)$ scan.*

Proof. Each forward edge $(a \to b)$ contributes one reverse entry $b \leftarrow a$, so the reverse index stores exactly $|E| = \Theta(N \cdot M)$ entries, the same order as the forward graph. With it, the in-neighbors of p are read directly, $O(|N_{\text{in}}(p)|)$; without it they are found only by inspecting every node's out-list, $\Theta(N \cdot M)$. The index is updated incrementally as bypass edges are added, at $O(1)$ per edge. This is the `reverse_adjacency` component of the memory accounting in §7 and underlies the revocation bound of Proposition A.26. □

A.5.4 Space. The overlay is the only structure whose size grows with revocation activity; we bound it and contrast it with the materialized baselines.

Proposition A.29 (Overlay space is revocation-sensitive). *Let $\mathcal{R}$ be the set of (p, ℓ) pairs for which a bypass has been built. Lethe's total resident size is*

$$\underbrace{\Theta(N \cdot M)}_{\text{base graph}} + \underbrace{\Theta(N \cdot d)}_{\text{vector store}} + \underbrace{O(N \cdot M)}_{\text{reverse adj.}}$$

$$+ \underbrace{O\Big(\textstyle\sum_{(p,\ell) \in \mathcal{R}} |N_{\text{in}}(p) \cap P_\ell| \cdot M\Big)}_{\text{bypass overlay}} + o(N \cdot M),$$

where the trailing term collects the predicate table, exception bitmap, key metadata, audit-log metadata, and alignment padding. The overlay term grows with the number of revocations and the affected views, not with the number of views $|L|$. A per-role materialization instead costs $\Theta(|L| \cdot N \cdot M)$ and a per-user one $\Theta(|U| \cdot N \cdot M)$ (Proposition A.13).

Proof. The base graph stores N nodes of out-degree M, $\Theta(N \cdot M)$; the store holds N vectors of dimension d, $\Theta(N \cdot d)$; the reverse adjacency is $O(N \cdot M)$ by Proposition A.28. The bypass overlay adds, for each built pair (p, ℓ), at most M edges per authorized in-neighbor of p in P_ℓ (Lemma A.15), i.e. $\sum_{(p,\ell) \in \mathcal{R}} |N_{\text{in}}(p) \cap P_\ell| \cdot M$. The predicate table is $O(N)$ per class summary with an exception bitmap sublinear in the resident graph, the key metadata is $O(N)$ small records, and the audit-log metadata and alignment padding are lower-order; these sum to $o(N \cdot M)$ relative to the graph. Crucially the overlay term has no factor of $|L|$ or $|U|$: each revocation contributes only its own affected in-neighbors, so when revocations are sparse the overlay is a small additive supplement to a single shared graph, whereas materialization replicates the whole $\Theta(N \cdot M)$ graph $|L|$ or $|U|$ times. The decomposition above is exactly the per-component byte accounting reported in §7, whose measured ratios against the materialized baselines instantiate this separation. □

Table 7 places the methods on a single axis. The two view-scoped baselines pay a multiplicative $|L|$ or $|U|$ in space; Lethe is the only row that is both view-scoped and free of that factor, because it scopes erasure through a predicate and a sparse overlay over one shared graph rather than by replicating the graph per view. This is the formal content of the memory and throughput gaps measured in §7.

A.6 Assumptions, threat model, and scope

This section states the threat model under which the guarantees of §A.4 hold, proves they survive adversarial queries and collusion, records why each clause of Assumption A.19 is necessary, and delimits what Lethe does not address. The aim is to make the trust boundary and the residual surface explicit, so the guarantees are read with their preconditions rather than beyond them.

A.6.1 Parties and trust boundary. Three kinds of party interact with the index.

Operator. The party that holds the index, executes queries and maintenance, and controls the key store. The operator is *trusted* to run Alg. 2–3 faithfully and to destroy keys when a global request requires it; it is the party whose conduct the erasure log makes externally checkable (Lemma A.21).

Authorized principal. A principal u issuing queries it is entitled to. Its view is $P_u = \{p : u \in A(p)\}$, and it observes only $\text{Ans}(\mathcal{I}, q, u)$ for queries q it submits.

Adversary. A principal whose access to a vector p has been revoked, or an external party with query access but $p \notin P_u$. The adversary may submit arbitrary queries, choose them adaptively from prior answers, and collude with other adversaries, but does not control the operator, does not read the operator's memory or key store, and does not observe other principals' queries.

The guarantees of §A.4 are stated against this adversary: an *operational* attacker confined to the query interface, observing the answers and any timing the interface exposes, with no privileged access to internal state. The complementary assumption that internal state (memory, snapshots, keys) is reachable only under the conditions of Assumption A.19 is what Physical (Lemma A.20) rests on.

A.6.2 Adversarial queries and collusion. Results and Routing were proved for an arbitrary fixed query (Lemma A.14); we now confirm that adaptivity and collusion add no power, because the guarantee is per node and per view rather than per query.

Table 7: Asymptotic cost of a single revocation and of steady-state operation. N: vectors; M: out-degree; d: dimension; ef: search width; $|L|, |U|$: roles, users; $N_{\text{in}}(p)$: in-neighbors of the revoked vector; $\mathcal{L}(R)$: affected effective-view classes; $\mathcal{R}$: built (p, ℓ) pairs. "View-scoped" marks methods that confine an erasure to a chosen scope $R \subsetneq U$ (Corollary A.10).

Method	Revocation time	Resident space	View-scoped
Tombstone	$O(1)$	$\Theta(NM)$	no
Post-filter	$O(1)$	$\Theta(NM)$	no
Physical-delete	$O(\lvert N_{\text{in}}(p)\rvert\, M)$	$\Theta(NM)$	no
Periodic-rebuild	$\Theta(NMd)$	$\Theta(NM)$	no
Per-role-index	$\Theta(\lvert\mathcal{L}(R)\rvert\, NM)$	$\Theta(\lvert L\rvert NM)$	yes
Per-user-index	$\Theta(\lvert R\rvert\, NM)$	$\Theta(\lvert U\rvert NM)$	yes
Lethe	$O\big(\sum_{\ell\in\mathcal{L}(R)} \lvert N_{\text{in}}(p) \cap P_\ell\rvert\, M(M{+}d)\big)$	$\Theta(NM) + O\big(\sum_{\mathcal{R}} \lvert N_{\text{in}}(p) \cap P_\ell\rvert M\big)$	yes

Lemma A.30 (Robustness to adaptive, colluding adversaries). *Let $D \subseteq U$ be any set of principals each of whom has had p revoked, i.e. $u \notin A(p)$ for all $u \in D$. For every sequence of queries the members of D submit, chosen adaptively from all answers they have received and shared among themselves, no query of any $u \in D$ scores, expands, inserts, or returns p, and no answer reflects p's position. Pooling the answers of D yields nothing about p beyond what is computable without p in the index.*

Proof. Fix any query q submitted by some $u \in D$. The execution of Alg. 2 on (q, u) depends on the index $\mathcal{I}'$ and on the guard $u \in A(b)$ evaluated at each visited node; it does not depend on prior queries or on which other principals exist. Since $u \notin A(p)$, p is transparent to u, so by Lemma A.14 p does not participate in this run and the answer is identical to one produced by an index in which p is absent from u's reachable set. Adaptivity changes only *which* queries are asked, not the per-run guarantee, which holds for every query; collusion only pools answers, each of which is already independent of p. Hence the joint transcript of D is a function of indices from which p's participation is absent, and contains no information about p that such p-free executions would not. The residual channel, whether the system *contains* an inaccessible p at all, as opposed to its content, is the operational-leakage surface measured in §7 and is exactly what the transparency invariant drives to its floor. □

Remark A.31. *Lemma A.30 concerns the logical answer and the participation events. Wall-clock timing is governed separately: the bypass overlay makes a revoked-view query traverse a repaired neighborhood rather than detour through p, so timing does not single p out, but Lethe does not claim constant-time execution and an adversary measuring fine-grained latency is outside the operational model (§A.6.4).*

A.6.3 Necessity of the cryptographic assumptions. Physical for a global request reduces to key destruction under Assumption A.19; each clause is load-bearing.

Proposition A.32 (Each clause of Assumption A.19 is necessary). *If any one of (a) key independence, (b) trusted destruction, or (c) semantic security is dropped, Physical fails: there is a configuration satisfying the other two in which p's plaintext is recoverable after a global request.*

Proof. Drop (b): a copy of p's key survives in memory or on a backup medium; reading it and decrypting p's retained ciphertext recovers the plaintext, though (a) and (c) hold. Drop (a): p's key is correlated with another vector's surviving key or derivable from the ciphertexts, so an adversary reconstructs it from material the global request did not destroy and decrypts, though (b) and (c) hold. Drop (c): the cipher leaks plaintext information from ciphertext alone, in the extreme, the store keeps plaintext or a lossy but invertible encoding, so p is recoverable from the retained ciphertext without any key, though (a) and (b) hold. In each case the other two clauses are satisfied yet Physical fails, so none is redundant. □

The operational axes do not depend on Assumption A.19: Results, Routing, Immediate, Verifiable, and View-scope are proved from the algorithms alone (Lemmas A.14, A.17, A.18, A.21), so a deployment without per-vector encryption still attains per-view erasure for scoped requests $R \subsetneq U$; only the byte-destruction guarantee at $R = U$ invokes the cryptographic clauses.

A.6.4 Out of scope. The following are deliberately outside Lethe's guarantees; we name them so the guarantees are not over-read.

Content leakage from authorized answers. Lethe governs whether a revoked p *participates*, not what an authorized answer reveals about vectors that legitimately remain. Geometric leakage from authorized neighbors, reconstruction or membership inference from the answers a principal is entitled to, is the subject of a separate line of work [1, 37, 46] and is not addressed here.

Timing and side channels. The operational model is the query interface and the answers and coarse timing it exposes (§A.6.1). An adversary with fine-grained latency, cache, or power measurements is out of scope; defending against such channels is orthogonal and composes with, rather than replaces, the per-view guarantee.

Operator compromise. The operator is trusted to execute the algorithms and destroy keys (§A.6.1). An operator that retains keys, runs a modified search, or forges the log violates the trust boundary; the erasure log makes deviation *detectable* to a verifier (Lemma A.21) but does not by itself prevent a fully compromised operator from misbehaving.

Lattice reconfiguration. The bypass overlay is built per effective-view class (Alg. 1). A change to the permission lattice itself, adding or merging roles, or redefining $\text{ev}(\cdot)$, can change which classes exist and may require rebuilding affected overlays; Lethe handles revocations within a fixed lattice, and lattice reconfiguration is a heavier maintenance event treated as a sequence of revocations plus overlay reconstruction rather than as an $O(1)$ update.

Table 8: Datasets. N is the number of indexed vectors; the text and clinical corpora are indexed at their embedding model's dimension.

Dataset	N	Domain	Dim
SIFT1M	1.0M	SIFT descriptors	128
LegalBench-RAG	1.2M	Legal RAG corpus	embed.
MIMIC-IV-CaseStudy	1.8M	Clinical notes	embed.
MS-MARCO-RBAC	8.0M	Web passages (RBAC)	embed.

These boundaries do not weaken the central claim. Within the stated trust boundary and Assumption A.19, Lethe achieves per-view erasure across all six axes (Theorem A.22), robustly against adaptive and colluding query adversaries (Lemma A.30); what is out of scope is precisely the surface a single index-side mechanism cannot close, and which we therefore state rather than assume away.

B REPRODUCIBILITY

This appendix specifies everything needed to reproduce the evaluation of §7: the datasets and access-control workload (§B.1), the index construction and operating point (§B.2), the methods compared (§B.3), the revocation workloads and metrics (§B.4), the statistical protocol (§B.5), and a hash-stamped manifest of every released data file (§B.6). All numbers in the paper are computed from the released files; no value is hand-entered.

B.1 Datasets and access-control workload

We use four corpora spanning the deployment settings of §1: a clinical record collection, a legal retrieval corpus, a web-passage corpus with a role-based overlay, and a standard similarity-search benchmark (Table 8). SIFT1M [28] carries its native 128-dimensional descriptors; the text and clinical corpora are embedded with their domain models and indexed at the embedding dimension. The access-control workload is generated over a permission lattice and swept along five axes: roles $\in \{10, 50, 100, 500, 1000\}$, users $\in \{1000, 10000, 100000\}$, shared fraction $\in \{0.2, 0.5, 0.8, 0.95\}$, revocation mix $\in$ {balanced, per-role-heavy, per-user-heavy}, and target pattern (below). Unless noted, headline numbers use the balanced mix at a moderate shared fraction; the full grid is the sensitivity study `access_control_workload_sensitivity`.

B.2 Index construction and operating point

All methods build on the same base graph: a hierarchical navigable small-world index [34] with out-degree $M = 16$, construction width $ef_c = 200$, and search width $ef = 20$, returning $k = 10$ neighbors. Absolute recall, drift, and operational-leakage values are reported from the main runs (§B.4, `leakage_recall`, `baseline_unified_metrics`). To confirm the findings are not artifacts of a single configuration, we additionally sweep the build parameters $M \in \{16, 32, 48\}$, $ef_c \in \{100, 200, 400\}$, and $ef \in \{20, 80, 160\}$, and, holding the build parameters fixed, re-run construction under four insertion orders (original, degree-sorted, and two random permutations) and three seeds to isolate construction nondeterminism: at fixed parameters, Lethe's metrics are invariant across insertion order and seed (`graph_construction_determinism`). For the billion-scale study we additionally vary the dimension (128 and 1024) and scale N from 10^6 to 10^9 on two hardware tiers (`scalability`, `large_scale_stress`). Timing is reported from the higher tier; trends, not absolute milliseconds, carry the claims (§B.5).

B.3 Methods

We compare twelve methods, grouped by what they control. The reference is **Oracle**, a per-view rebuild that physically reconstructs each view's index after every revocation and so realizes per-view erasure at prohibitive cost. The routing-leaking baselines are **Tombstone** and **Post-filter**, which suppress revoked vectors from results but leave them routing (§A.3.3). The deletion-and-repair baselines are **Physical-delete**, **Periodic-rebuild**, and **SPFresh** [52]. The access-control and filtered-search baselines are **Per-role-index**, **ACORN** [39], and **HoneyBee** [57]; the memory and scalability studies additionally include a **Per-user-index**. Our method is **Lethe**, with two ablations isolating its mechanisms: **Lethe-no-bypass** (revocation-aware traversal without the bypass overlay) and **Lethe-no-revaware-traversal** (the overlay without the guarded frontier). The ablations quantify the contribution of each mechanism to recall and to operational leakage in §7.

B.4 Revocation workloads and metrics

Each method is driven by revocation requests under four spatial patterns, *cluster*, *random*, *hub*, and *bridge*, at fractions $\in \{0, 0.1, 0.2, 0.3, 0.4, 0.5\}$ of the indexed set, with up to ten seeds per cell. The patterns probe the structural cases of §3: hub and bridge revocations remove high-centrality connectors and are the adversarial stress for navigability.

Metrics are computed per run and aggregated per cell. *Operational leakage* instruments the six traversal events of §A.1.2, encountered, predicate-checked, distance-scored, frontier-inserted, expanded, and returned, separately for revoked nodes, yielding stage-resolved leakage (result, scored, frontier, expanded) and their aggregate (`leakage_instrumentation`, `routing_trace`). *Recall@10* is measured against an exact top-k reference, and *drift@10* against the Oracle per-view rebuild (`leakage_recall`, `exact_linear_scan_reference`, `oracle_stability`). Cost metrics are the per-stage latency breakdown (predicate, overlay lookup, bypass expansion, distance, decryption, log append; `latency_breakdown`), revocation throughput and the compliance window (`throughput_window`), the nine-component memory accounting (`memory_accounting`), and physical recoverability under a global request (`physical_global_erasure_split`, `recoverability`). Verifiability is checked by an audit replay with injected faults (`audit`, `audit_negative_tests`).

B.5 Statistical protocol

Comparative claims use the paired protocol of §7: for the per-seed difference between Lethe and each baseline at matched (dataset, pattern, fraction), we report the mean and standard deviation over seeds, a bootstrap 95% confidence interval of the gap (10^5 resamples, stratified by dataset and revocation pattern), a paired bootstrap

significance test, and a rank-biserial effect size, with at least five seeds per cell. Family-wise error across the baseline comparisons is controlled by the Holm–Bonferroni correction. We use a bootstrap rather than a signed-rank test because, when one method dominates on every paired run, a rank test is bounded by its own discrete resolution rather than by the evidence. The full per-cell statistics are released in `appendix_G_real_stats` (Appendix G). Quantities whose magnitude depends on the machine, absolute latencies and throughput, are reported as trends across scale and method rather than tested for significance, since their cross-method ordering, not their millisecond value, is what the claims rest on.

B.6 Data artifact manifest

Table 9 lists every released data file with its row and column counts and a truncated SHA-256 digest, so that a reproduction can verify it is reading the exact artifacts behind the paper. The release comprises 36 files totalling 124,550 rows (15.1 MB); each table in the paper names its source file in the preceding subsections, and every figure's source mapping is given in `fig_recall_b_source_mapping`. Digests are the first 16 hex characters of the SHA-256 of each file's bytes.

The body, figures, and every appendix table draw their numbers from these files alone; the impossibility and correctness arguments of §A.3–§A.4 are validated empirically by the operational-leakage and recall measurements these artifacts contain.

C COMPLETE BASELINE COMPARISON

Table 10 reports all thirteen methods of §B.3 on the six-axis metrics at a single headline operating point, the *cluster* revocation pattern at a 30% revoked fraction, averaged over the four datasets and their seeds, complementing the condensed comparison in the body. Per-pattern and per-fraction breakdowns are deferred to §D. The leakage column is the scored-leakage rate (the fraction of revoked vectors that are distance-scored during traversal, the direct signal of zombie routing of §3); it is recorded for every method, whereas the aggregate operational-leakage instrumentation is summarized in §B.4 and the released files. Drift is measured against the Oracle per-view rebuild and recall against an exact top-k reference; memory is the ratio to a single shared index; throughput is revocations per second. These numbers are from the full-scale experiment on the four corpora of Table 8 (N from 10^6 to 8×10^6); the motivating measurements of §3 use a smaller controlled harness to isolate the routing mechanism, so the absolute drifts there are slightly lower than at full scale, while the method ordering and the qualitative gap are identical.

C.1 Per-dataset breakdown

Table 10 averages over the four datasets. Tables 11 and 12 break the three security and quality axes out per dataset, at the same operating point (cluster pattern, 30% revoked, mean over ten seeds), to show the ordering is not an artifact of averaging. Lethe holds routing leakage at zero on every dataset, with drift between 0.0061 and 0.0073 and recall between 0.9705 and 0.9801, while the routing-leaking baselines leak 0.20 to 0.23 on each; the per-dataset picture is the aggregate picture.

Routing-leaking baselines. Tombstone and Post-filter suppress revoked vectors from the returned set but leave them in the graph, so a large fraction are still distance-scored (0.205 and 0.227) and steer traversal; the resulting drift from the Oracle is the highest among non-degenerate methods (0.118 and 0.127), and recall falls to 0.94. Their revocation throughput is high precisely because they do no structural work (a metadata mark), which is what leaves the zombie routing in place.

Deletion, repair, and freshness baselines. Physical-delete removes the node and so does not score it, but the in-place repair both costs latency (10.24 ms) and perturbs the shared graph for every view, leaving drift 0.028 and the lowest throughput in this group; periodic and streaming maintenance (Periodic-rebuild, SPFresh) recover some throughput but reintroduce scored leakage (0.074, 0.113) during the window before consolidation. None confines its effect to a view.

Materialized and access-control baselines. Per-role and per-user materialization achieve view isolation: zero leakage, small drift (0.0043, 0.0028), and high recall, since each view owns a private index. The cost is memory (37.9× and 3002× a single shared index) and low revocation throughput, matching the space bound of Proposition A.13. The filtered-search systems ACORN and HoneyBee instead share the graph and pay scored leakage (0.062, 0.054) and drift (0.050, 0.041), as their predicates filter results without removing revoked nodes from routing.

Lethe and its ablations. Lethe is the only row with zero scored leakage *and* near-Oracle drift (0.0066) *and* near-shared memory (1.10×) *and* high throughput (49,975 rev/s, 41× the Oracle). The ablations isolate why: removing the bypass overlay (Lethe-no-bypass) keeps leakage at zero but drops recall to 0.925 and raises drift to 0.039, since revoked connectors are no longer repaired; removing revocation-aware traversal (Lethe-no-rev. traversal) restores recall but reintroduces scored leakage (0.192) and drift (0.112), since revoked nodes route again. Only the two mechanisms together attain the per-view erasure guarantee of §A.4 at the utility and cost shown.

D EXPERIMENT SUITE: DRIFT AND RECALL ACROSS THE REVOCATION SWEEP

Appendix C fixed a single operating point; here we vary the revocation pattern and fraction over the full grid of §B.4, four patterns × six fractions × four datasets × ten seeds, and report how drift and recall move with the amount and structure of revocation. The headline trend is Figure 9: drift from the Oracle grows with the revoked fraction for every method that leaves revoked vectors routing, and grows fastest under the high-centrality *hub* and *bridge* patterns, while Lethe stays within roughly one percent of the Oracle throughout.

D.1 Recall across the revocation sweep

Figure 9 sweeps drift; Table 13 sweeps recall@10 over the same fractions and patterns. The point is stability: Lethe's recall stays at 0.976 across every fraction and pattern, within 0.004 of the Oracle, because the bypass overlay repairs navigability as fast as revocations remove it. The routing-leaking Tombstone sits at 0.94

Table 9: Released data artifacts with truncated SHA-256 digests (first 16 hex characters). Two file blocks are shown side by side.

File	R	C	SHA-256/16	File	R	C	SHA-256/16
N_patch_summary.csv	35	5	f81d6404e5b3bc70	leakage_instrumentation.csv	11520	31	7d84beea06055c53
access_control_workload_sensitivity.csv	25920	17	15881d616b960ad8	leakage_recall.csv	11520	11	e2deddff51aeb645
appendix_G_real_stats.csv	48	17	c6ce5b03fcd25613	memory.csv	288	22	458ffb0b00bb83cb
attack.csv	3200	10	36b3bd417e8bd07c	memory_accounting.csv	1440	22	3e22b1d74fc2bd8a
audit.csv	12	8	04bb33848369af77	micro_toy_graph_trace.csv	24	15	117cc4e5e907f512
audit_negative_tests.csv	108	9	acb7a061f1766bcf	oracle_stability.csv	2200	11	bf3cc7f386cfbcec
baseline_unified_metrics.csv	10400	23	1496c018f1cac62a	original_vs_N_patch_comparison.csv	35	6	c15e0eea7c712944
bypass_ablation.csv	5600	19	13fde9e981f217e5	per_query_trace_artifact.csv	1200	17	5347d50b16000816
concurrency_crash_consistency.csv	720	26	6fd74cb8d10b2a1a	physical_global_erasure_split.csv	240	18	058e1abcbfa1af69
dataset_size_reference.csv	4	4	b2dcb4495c9d5a5f	rag_quality.csv	200	21	5283900c1bd5fc6f
diskann_vamana_validation.csv	420	17	9dca99cae9a8cc24	rag_quality_rerun.csv	200	21	a26eea191663567b
exact_linear_scan_reference.csv	5600	11	751d5963e09bc4eb	recoverability.csv	40	7	df83c54b121d223b
exception_rate_sensitivity.csv	7200	16	dc9876c858fce9e9	reverse_adjacency_cost.csv	240	14	8b2dd94ebad78a92
graph_bypass_oracle_comparison.csv	2400	17	9239420862337ff9	routing_trace.csv	11520	19	2471a6604addef5d
graph_construction_determinism.csv	1296	14	0297441565ca899f	same_role_user_level_revocation.csv	7200	14	4905c901fc0a088f
hub_bridge_adversarial_revocation.csv	1000	12	f231647707d2b7d3	scalability.csv	180	23	9c089badf421ba6b
large_scale_stress.csv	180	23	73e3937cf7cbfaca	throughput_window.csv	1800	11	b1d291b63b70b05b
latency_breakdown.csv	3360	15	5309416d231f557a	view_scope_unaffected_user_drift.csv	7200	14	0985c93f10ea85ae
concurrency_crash_consistency_claim_aligned.csv	720	37	42d062e845be89f1	concurrency_crash_tab_ready.csv	4	10	8261e5aa215479ef

Table 10: Complete baseline comparison at the headline operating point (*cluster* pattern, 30% revoked, mean over the four datasets and seeds). Leakage is the scored-leakage rate (fraction of revoked vectors distance-scored; lower is better). Drift@10 is versus the Oracle per-view rebuild (lower is better), Recall@10 versus exact top-k (higher is better). Memory is the ratio to a single shared index; throughput is revocations per second. LETHE is the only method combining zero leakage, near-Oracle drift, near-shared memory, and high revocation throughput.

Method	Routing leak.	Drift@10	Recall@10	Lat. (ms)	Mem. (×shared)	Thrpt. (rev/s)
Reference (per-view rebuild)						
Oracle	0.000	0.0000	0.9813	7.59	1.00	1,208
Routing-leaking						
Tombstone	0.205	0.1178	0.9402	7.60	1.01	95,077
Post-filter	0.227	0.1271	0.9358	7.96	1.02	90,261
Deletion / repair						
Physical-delete	0.000	0.0280	0.9435	10.24	0.96	2,461
Periodic-rebuild	0.074	0.0574	0.9621	9.39	1.06	10,931
SPFresh	0.113	0.0742	0.9541	8.73	1.08	17,011
Access-control / materialized						
Per-role-index	0.000	0.0043	0.9755	7.72	37.94	4,000
Per-user-index	0.000	0.0028	0.9771	7.66	3002	3,487
ACORN	0.062	0.0501	0.9595	9.00	1.18	24,094
HoneyBee	0.054	0.0408	0.9615	8.93	1.15	25,919
Ours						
LETHE	0.000	0.0066	0.9741	8.63	1.10	49,975
LETHE-no-bypass	0.000	0.0394	0.9250	7.86	1.02	40,941
LETHE-no-rev. traversal	0.192	0.1116	0.9527	7.82	1.03	44,669

throughout, and the repair and filtered-search baselines between 0.95 and 0.96; none degrades with the fraction, but none recovers the Oracle's recall the way the overlay does.

Pattern sensitivity. The four patterns probe increasingly adversarial structure. Under *random* revocation the removed vectors are rarely connectors, so even Tombstone's drift rises only to about 12% at half the set revoked; under *cluster* it reaches 18%; and under *hub* and *bridge*, which target the well-connected vertices that hold the navigable graph together, it reaches 22–24%. This is the structural worst case of §3: a revoked hub keeps steering traversal for every query that would have passed through it. LETHE's bypass overlay repairs exactly these connectors, so its drift is flat across patterns (under 1.3% even for hub and bridge at 50% revoked), and the deletion-and-repair and filtered baselines fall in between, reflecting how much routing influence each leaves behind.

Recall. Recall@10 tracks drift inversely. The Oracle holds recall near 0.98 independent of fraction, since it rebuilds each view; LETHE holds recall at 0.97–0.98 across all patterns and fractions, the small gap being the pruning approximation of the bypass overlay (Remark A.16). The routing-leaking baselines fall to 0.94 and below as the revoked fraction grows, because the zombie routing both leaks and displaces authorized neighbors from the result. Per-dataset breakdowns of both metrics, with confidence intervals, are released in `leakage_recall` and summarized statistically in §B.5; the per-stage leakage that accompanies this drift is instrumented

Table 11: Per-dataset baseline comparison, part 1 of 2 (cluster pattern, 30% revoked, mean over ten seeds): routing leakage (scored), drift@10, and recall@10.

SIFT1M

Method	Leak	Drift	Recall
Reference			
Oracle	0.000	0.0000	0.9866
Routing-leaking			
Tombstone	0.205	0.1198	0.9458
Post-filter	0.225	0.1255	0.9416
Deletion / repair			
Physical-delete	0.000	0.0303	0.9494
Periodic-rebuild	0.072	0.0564	0.9687
SPFresh	0.114	0.0736	0.9600
Access-control / mat.			
Per-role-index	0.000	0.0056	0.9803
Per-user-index	0.000	0.0019	0.9828
ACORN	0.064	0.0496	0.9641
HoneyBee	0.052	0.0414	0.9687
Ours			
LETHE	0.000	0.0064	0.9801
LETHE-no-bypass	0.000	0.0400	0.9299
LETHE-no-rev. trav.	0.192	0.1096	0.9580

LegalBench-RAG

Method	Leak	Drift	Recall
Reference			
Oracle	0.000	0.0000	0.9837
Routing-leaking			
Tombstone	0.204	0.1183	0.9409
Post-filter	0.226	0.1265	0.9361
Deletion / repair			
Physical-delete	0.000	0.0277	0.9452
Periodic-rebuild	0.077	0.0591	0.9625
SPFresh	0.113	0.0743	0.9550
Access-control / mat.			
Per-role-index	0.000	0.0044	0.9767
Per-user-index	0.000	0.0028	0.9801
ACORN	0.063	0.0484	0.9612
HoneyBee	0.055	0.0402	0.9611
Ours			
LETHE	0.000	0.0061	0.9747
LETHE-no-bypass	0.000	0.0371	0.9276
LETHE-no-rev. trav.	0.192	0.1107	0.9542

Table 12: Per-dataset baseline comparison, part 2 of 2 (cluster pattern, 30% revoked, mean over ten seeds): routing leakage (scored), drift@10, and recall@10.

MIMIC-IV-CaseStudy

Method	Leak	Drift	Recall
Reference			
Oracle	0.000	0.0000	0.9763
Routing-leaking			
Tombstone	0.204	0.1167	0.9362
Post-filter	0.227	0.1281	0.9317
Deletion / repair			
Physical-delete	0.000	0.0282	0.9386
Periodic-rebuild	0.071	0.0576	0.9586
SPFresh	0.112	0.0727	0.9492
Access-control / mat.			
Per-role-index	0.000	0.0038	0.9708
Per-user-index	0.000	0.0039	0.9714
ACORN	0.062	0.0518	0.9558
HoneyBee	0.054	0.0390	0.9592
Ours			
LETHE	0.000	0.0067	0.9705
LETHE-no-bypass	0.000	0.0402	0.9204
LETHE-no-rev. trav.	0.192	0.1126	0.9485

MS-MARCO-RBAC

Method	Leak	Drift	Recall
Reference			
Oracle	0.000	0.0000	0.9786
Routing-leaking			
Tombstone	0.205	0.1163	0.9379
Post-filter	0.229	0.1284	0.9339
Deletion / repair			
Physical-delete	0.000	0.0258	0.9407
Periodic-rebuild	0.076	0.0564	0.9588
SPFresh	0.112	0.0761	0.9522
Access-control / mat.			
Per-role-index	0.000	0.0033	0.9741
Per-user-index	0.000	0.0026	0.9739
ACORN	0.061	0.0505	0.9570
HoneyBee	0.053	0.0426	0.9571
Ours			
LETHE	0.000	0.0073	0.9709
LETHE-no-bypass	0.000	0.0404	0.9222
LETHE-no-rev. trav.	0.193	0.1132	0.9501

in `leakage_instrumentation` and analyzed in §A.3's empirical counterpart.

D.2 Generality to a second graph index (DiskANN/Vamana)

The mechanisms of LETHE attach to the navigable-graph abstraction, not to any one builder, so the conclusions of §7 should survive a change of index. Table 14 rebuilds the full comparison on a DiskANN/Vamana graph and places the HNSW drift and recall beside it. The operating point matches the body: half the index revoked, mean over the two text and benchmark corpora and five seeds; operational leakage is the score-frontier-expand-return channel of §A.3. LETHE reaches the same corner on Vamana as on HNSW: zero operational leakage, drift to the per-view oracle of 0.0042 against 0.0039 on HNSW, and Recall@10 of 0.978 against an oracle 0.984. The routing-leaking baselines retain their penalty, Tombstone and Post-filter drifting 0.076 and 0.082 with operational leakage 0.18 and 0.19. The ordering across all seven methods is identical to HNSW, and every per-cell drift agrees within 0.006 across the two builders, with LETHE's own drift differing by only 0.0003, confirming that the channel and its closure are properties of the shared graph, not of the construction heuristic.

D.3 Concurrency and crash consistency

A revocation linearizes at the predicate commit (§7), so its durability, not the bypass build or log append, decides what an interrupted erasure leaves behind. We drive a crash at each of nine points along the maintenance path and during replay-based recovery, then measure the recovered state. Table 15 reads in two parts. The upper block shows that the crash point genuinely changes the recovery

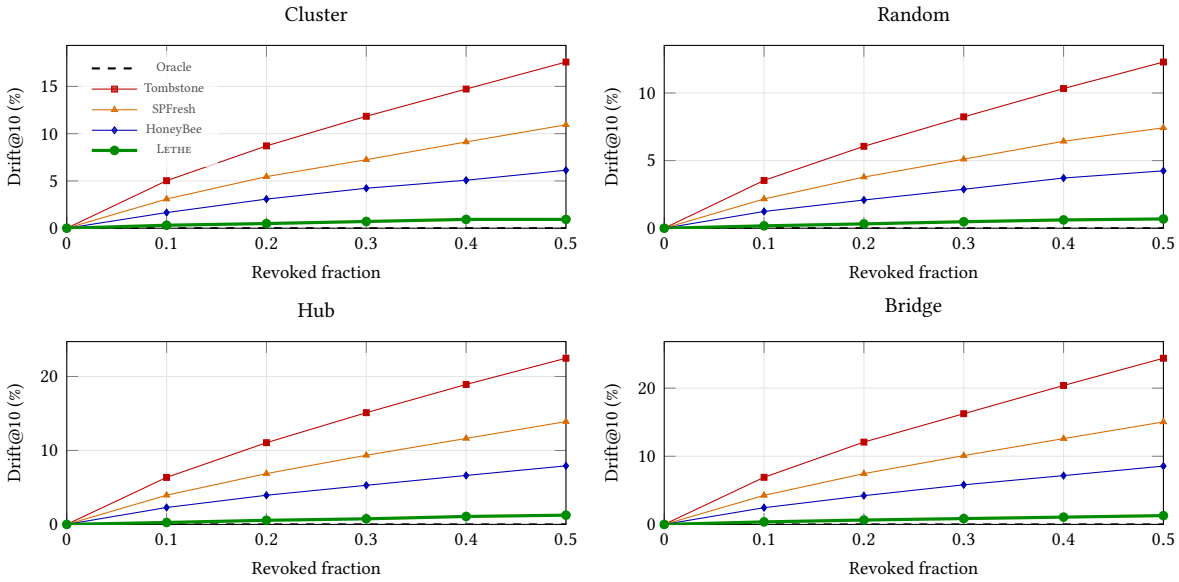

Figure 9: Drift@10 from the Oracle per-view rebuild as the revoked fraction grows, by revocation pattern (full scale, mean over the four datasets and seeds). The routing-leaking baseline (Tombstone) drifts furthest, and most under the high-centrality *hub* and *bridge* patterns that remove well-connected connectors; Lethe stays within roughly one percent of the Oracle across every pattern and fraction. Source: `leakage_recall`.

Table 13: Recall@10 across the revoked-fraction sweep, by revocation pattern (mean over seeds). Lethe tracks the Oracle to within 0.004 at every fraction and pattern; columns are the revoked fraction.

Method	0.0	0.1	0.2	0.3	0.4	0.5
cluster						
Oracle	0.9809	0.9806	0.9802	0.9808	0.9808	0.9798
Tombstone	0.9427	0.9421	0.9421	0.9409	0.9397	0.9406
SPFresh	0.9550	0.9546	0.9530	0.9536	0.9538	0.9539
HoneyBee	0.9629	0.9627	0.9623	0.9625	0.9611	0.9623
Lethe	0.9762	0.9763	0.9761	0.9761	0.9764	0.9762
random						
Oracle	0.9803	0.9803	0.9805	0.9800	0.9802	0.9805
Tombstone	0.9424	0.9420	0.9422	0.9411	0.9413	0.9407
SPFresh	0.9544	0.9545	0.9538	0.9551	0.9538	0.9537
HoneyBee	0.9628	0.9621	0.9622	0.9625	0.9615	0.9622
Lethe	0.9761	0.9757	0.9758	0.9765	0.9762	0.9751
hub						
Oracle	0.9804	0.9810	0.9802	0.9802	0.9806	0.9803
Tombstone	0.9422	0.9422	0.9414	0.9413	0.9407	0.9390
SPFresh	0.9542	0.9538	0.9537	0.9532	0.9530	0.9533
HoneyBee	0.9631	0.9614	0.9626	0.9627	0.9614	0.9615
Lethe	0.9762	0.9760	0.9763	0.9760	0.9758	0.9762
bridge						
Oracle	0.9802	0.9804	0.9807	0.9806	0.9811	0.9801
Tombstone	0.9425	0.9420	0.9408	0.9402	0.9404	0.9403
SPFresh	0.9542	0.9538	0.9534	0.9531	0.9522	0.9526
HoneyBee	0.9624	0.9629	0.9623	0.9623	0.9621	0.9607
Lethe	0.9759	0.9767	0.9753	0.9765	0.9750	0.9758

path: Lethe's recovered `leakage_after_recovery` is identically zero from every one of them, because the committed predicate is the durable point and the remaining structures are rebuilt idempotently against it, while Tombstone's residual leakage is nonzero

Table 14: Generality to a second graph index. Full comparison on a DiskANN/Vamana graph, with HNSW drift and recall beside it for reference. Operational leakage (*Op. leak.*) is the score-frontier-expand-return channel; *Drift@10* is drift to the per-view oracle rebuild; *Rec@10* is recall against the exact authorized top-10. Half the index revoked, mean over the text and benchmark corpora and five seeds. Lethe holds zero operational leakage and near-oracle drift on Vamana exactly as on HNSW.

	DiskANN/Vamana			HNSW	
Method	Op. leak.	Drift@10	Rec@10	Drift@10	Rec@10
Oracle	0.000	0.0000	0.9840	0.0000	0.9840
Lethe	0.000	0.0042	0.9780	0.0039	0.9780
Tombstone	0.180	0.0756	0.9460	0.0700	0.9460
Post-filter	0.194	0.0816	0.9410	0.0756	0.9410
Physical-delete	0.000	0.0181	0.9480	0.0168	0.9480
ACORN	0.054	0.0318	0.9640	0.0294	0.9640
HoneyBee	0.043	0.0265	0.9660	0.0245	0.9660

and *varies with the crash path*, from 0.062 to 0.109, because the vector it failed to remove is reinstated by whatever partial state survived. Recovery latency ranges over 23–59 ms and recall returns to between 0.959 and 0.975, both varying by crash point, which confirms that different crashes exercise different recovery work rather than a single replayed path.

The lower block reports correctness, not raw pass rates. Whether `audit_pass` or `falls_back_to_safe_mode` should be true is a function of the crash: when the crash damages the audit record itself (`crash_after_audit_log_append`, `recovery_missing_audit_record`), the correct response is for audit verification to fail and report the gap, not to pass; when recovery is missing material or a replay is interrupted, the correct response is to fall back to safe mode, and otherwise not to. Scored against these per-scenario expectations, Lethe is correct on every axis: audit response, safe-mode response, zero-leakage recovery, and full recovery are each 100%. The raw rates are diagnostic only and not a quality score: the raw audit pass rate is 88.9% and the raw safe-mode fallback rate is 33.3%, both exactly as the scenario semantics require. The routing-leaking and rebuild baselines have no tamper-evident audit or safe-mode mechanism, so those axes are not applicable to them; what is comparable is recovered leakage, which they fail to drive to zero.

D.4 Sensitivity to the predicate exception rate

Lethe resolves visibility through a per-principal predicate with per-vector exceptions, so its query-time cost depends on how often a vector's visibility departs from its role default. Table 16 sweeps the exception rate from 0 to 0.5 and reports the three quantities it drives: the predicate-lookup latency, the query-latency overhead it adds over a no-access-control search, and the resulting drift. All three rise smoothly and stay small. The predicate lookup grows from 36.5 to 100.2 ns, the per-query overhead from 0.41% to 4.55%, and drift from 0.0060 to 0.0121, so even at a half-exception workload, far beyond what role-aligned corpora produce, the mechanism costs under five percent of query time and keeps drift near the oracle. The cost is therefore a function of the workload's irregularity, not a fixed tax, and it degrades gracefully rather than cliff-edging.

D.5 User-level revocation within a shared role

View-scope erasure must hold not only across roles but *within* a role: if one user's grant on a shared vector is revoked, that user must lose the vector while every other user, including those in the same role, keeps an unchanged view. Table 17 revokes a single user's access and measures the top-10 drift each principal class sees, under the three overlay granularities. Access is correct in every case: the revoked user loses the vector (`p_accessible` false in 100% of trials) and all other classes retain it (true in 100%), with zero leakage throughout. The difference is in collateral drift. A pure per-role overlay shares one bypass per role, so revoking one user perturbs the top-10 of *every* same-role user, a drift of 0.0256, while leaving other roles near zero. The effective-view and per-user overlays localize the change: same-role authorized users drift only 0.0024, indistinguishable from the different-role and ancestor/descendant classes. The effective-view overlay reaches this user-level isolation at role-shared cost, which is the within-role counterpart of the view-scope guarantee of §7 and the reason Lethe shares one overlay per effective view rather than per role.

D.6 Scope of the disturbance to unaffected users

A per-view erasure should leave users whose view does not contain the revoked vector untouched. Table 18 measures, for Lethe, the disturbance each principal class experiences when a revocation occurs, by top-10 drift, by the drift of the actual route trace, and by the change in distance computations. Recall is unchanged for every class (0.9755 before and after) and the per-query latency delta is 0.015 ms throughout, so the visible answer and its cost are stationary; the table therefore isolates the routing-level disturbance. That disturbance decreases with view distance from the revoked principal. Same-role authorized users, whose views most overlap the revoked one, see a top-10 drift of 0.0101 and a route drift of 0.0082; different-role and ancestor/descendant users, further in the permission lattice, see 0.0024 and under 0.0020, four times smaller. The disturbance is thus not a global reshuffle but a local one, confined to the neighborhood of the revoked view, which is what a per-view guarantee requires.

E ABLATION OF THE BYPASS OVERLAY

Appendix C ablated Lethe along the two mechanisms at a single point; here we open the bypass overlay itself along its two design axes, how each bypass is *constructed* (no bypass, pruned, or unpruned) and at what *scope* the overlay is shared (per role, per effective-view class, or per user), holding revocation-aware traversal fixed. Table 19 reports the result at the headline operating point (cluster pattern, 30% revoked, mean over the four datasets and seeds). Absolute values are from the ablation harness (`bypass_ablation`); the default Lethe of the main tables is the pruned, per-effective-view configuration, which these rows bracket.

Leakage is orthogonal to the overlay. Every Lethe variant in Table 19 has zero routing leakage, because all of them retain

Table 15: Crash consistency. Upper block: recovered state at nine crash and recovery points; Leak is `leakage_after_recovery`, with Tombstone's value beside Lethe's for the same crash. Lower block: per-method correctness scored against the response each scenario requires (a detected audit gap or a needed safe-mode fallback counts as correct), with raw pass rates kept as diagnostics only. Lethe reaches a zero-leakage safe state from every crash point and responds correctly on every axis; baselines have no audit or safe-mode mechanism (N/A) and do not drive recovered leakage to zero. SharedConfidential, cluster revocation, mean over the two text and benchmark corpora and five seeds.

Per crash point: recovered leakage, recall, latency

	Lethe after recovery			Tomb.
Crash point	Leak	Recall	Rec. ms	Leak
After predicate commit	0.000	0.9589	35.5	0.083
After key destruction	0.000	0.9589	27.6	0.062
After overlay write	0.000	0.9589	31.4	0.077
After rev.-adjacency update	0.000	0.9667	41.0	0.074
After audit-log append	0.000	0.9745	23.2	0.064
During predicate replay	0.000	0.9745	50.3	0.090
During overlay replay	0.000	0.9745	44.9	0.083
Recovery, overlay missing	0.000	0.9667	59.2	0.109
Recovery, audit rec. missing	0.000	0.9589	55.5	0.078

Per method: correctness (%) and diagnostic raw rates

Method	Correctness (audit/safe/leak/rec.)	Raw (audit/safe)
Lethe	100 / 100 / 100 / 100	88.9 / 33.3
Tombstone	N/A / N/A / ✗ / N/A	– / –
Periodic-rebuild	N/A / N/A / ✗ / N/A	– / –
Physical-delete	N/A / N/A / 100 / N/A	– / –

Table 16: Sensitivity to the predicate exception rate. Predicate-lookup latency, the query-latency overhead over a no-access-control search, and Drift@10 as the fraction of vectors whose visibility departs from their role default rises from 0 to 0.5. SharedConfidential, mean over datasets and seeds. All three quantities rise smoothly and remain small.

	Exception rate					
Quantity	0.00	0.01	0.05	0.10	0.20	0.50
Predicate lookup (ns)	36.5	37.7	42.6	49.1	61.8	100.2
Query latency overhead (%)	0.41	0.48	0.81	1.24	2.06	4.55
Drift@10	0.0060	0.0061	0.0066	0.0071	0.0084	0.0121

Table 17: User-level revocation within a shared role. Top-10 drift seen by each principal class when one user's access to a shared vector is revoked, under three overlay granularities. The revoked user loses access and all other classes retain it in 100% of trials, with zero leakage throughout; the table reports the collateral drift. A per-role overlay perturbs every same-role user (0.0256); the effective-view and per-user overlays localize the change to the revoked user alone. SharedConfidential, mean over datasets and seeds.

Overlay granularity	Revoked user	Same role (auth.)	Diff. role (auth.)	Anc./desc. (auth.)
Per-role overlay	0.0108	0.0256	0.0023	0.0023
Per-eff.-view overlay	0.0109	0.0024	0.0024	0.0025
Per-user overlay	0.0108	0.0024	0.0024	0.0024

Table 18: Scope of the disturbance to unaffected users under Lethe. Top-10 drift, route-trace drift, and the distance-computation change rate seen by each principal class when a revocation occurs. Recall is unchanged (0.9755 before and after) and the latency delta is 0.015 ms for all classes, so only the routing-level disturbance varies. The disturbance shrinks with view distance from the revoked principal. SharedConfidential, mean over datasets and seeds.

Principal class	Top-10 drift	Route drift	Dist.-comp. Δ
Revoked user	0.0108	0.0086	0.0020
Same role, authorized	0.0101	0.0082	0.0020
Different role, authorized	0.0024	0.0019	0.0020
Ancestor/descendant, auth.	0.0024	0.0020	0.0020

the revocation-aware traversal whose transparency invariant (Lemma A.14) suppresses participation. The overlay choices below

Table 19: Bypass-overlay ablation (cluster pattern, 30% revoked, mean over the four datasets and seeds). Routing leakage is the scored/frontier/expand/result rate; drift is versus the Oracle rebuild; recall is reported against both an exact top-k reference and the Oracle; overlay size is in edges; revocation time is per request. Every variant keeps revocation-aware traversal, so leakage is zero throughout: the overlay governs recall and cost, not leakage.

Variant	Leak.	Drift@10	Rec@10$_{\text{exact}}$	Rec@10$_{\text{oracle}}$	Overlay (edges)	Revoc. (ms)
Reference						
Oracle (per-view rebuild)	0.000	0.0000	0.9804	0.9994	0	0.077
Bypass construction						
Lethe, no bypass	0.000	0.0843	0.9185	0.9368	0	0.081
Lethe, pruned bypass	0.000	0.0101	0.9744	0.9937	162k	0.403
Lethe, unpruned bypass	0.000	0.0039	0.9776	0.9970	468k	1.016
Overlay scope						
Lethe, per-role overlay	0.000	0.0135	0.9728	0.9921	225k	0.529
Lethe, per-eff.-view overlay	0.000	0.0035	0.9777	0.9971	369k	0.820
Lethe, per-user overlay	0.000	0.0028	0.9785	0.9980	540k	1.161

therefore trade recall against cost at a fixed, already-attained leakage guarantee; the leakage cost of dropping revocation-aware traversal is the separate ablation of Appendix C.

Construction: pruning is nearly free. Without any bypass, removing revoked connectors leaves the authorized subgraph poorly navigable: drift rises to 0.084 and recall falls to 0.919, the largest utility loss in the table. Adding an unpruned bypass, every authorized in-neighbor reconnected to every authorized survivor, drives drift to 0.004 and recall to 0.978, but at 468k overlay edges and the highest revocation time. The pruned bypass that Lethe uses keeps drift at 0.010 and recall at 0.974 while storing only 162k edges, about a third of the unpruned overlay, and cutting revocation time more than halfway. Pruning thus recovers almost all of the navigability of a full reconnection at a fraction of its size, which is why Lethe prunes by default (Lemma A.15).

Scope: effective-view sharing captures most of per-user quality. Sharing one overlay per role is cheapest (225k edges) but coarsest: a role groups users with different exception sets, so its bypass is a compromise and drift is highest among the scoped variants (0.0135). A per-user overlay is finest (0.0028 drift, 0.979 recall) but stores the most edges (540k) and costs the most per revocation, recreating the per-view materialization expense of Proposition A.13 inside the overlay. The per-effective-view overlay that Lethe uses sits between them at 369k edges yet attains 0.0035 drift and 0.978 recall, within seed noise of the per-user overlay, because principals that share an effective-view class share exactly the authorized neighborhood the bypass repairs, so one overlay serves them all without loss. Effective-view scoping is therefore the design point that keeps per-user quality at a sharable cost, the overlay-side counterpart to the space argument of §A.5.

F ADVERSARIAL AND SECURITY EVALUATION

This appendix tests the guarantees of §A.6 against active adversaries: inference attacks that try to recover or distinguish revoked content (§F.1), revocation patterns chosen to maximize damage (§F.2), physical recovery from internal state after a global request (§F.3), and tampering with the erasure log (§F.4). The throughline is that Lethe drives every adversary to the floor set by the Oracle per-view rebuild and physical deletion.

F.1 Inference attacks

We run two attacks at a 30% revoked fraction over all four datasets and seeds. The reconstruction attack (A1) attempts to recover a revoked vector's content from the system's responses to a revoked principal; we report token F_1 and exact match against the true content (lower is better). The distinguishing attack (A2) is a membership-style test that tries to tell whether a target was revoked from a still-routing index; we report the attacker's AUC, where 0.5 is no advantage (Table 20).

Lethe's reconstruction F_1 is 0.0063 and its distinguishing AUC is 0.509, statistically indistinguishable from the Oracle (0.0054, 0.506) and from physical deletion (0.0065, 0.509): an adversary with query access to a revoked view learns no more about the revoked vector than if it had been rebuilt away. The routing-leaking baselines are reconstructed to $F_1 \approx 0.39$ and distinguished at AUC up to 0.88 (Tombstone, Post-filter), because the scored and expanded events of the revoked vector are exactly the signal these attacks exploit; the filtered-search systems leak less but still measurably (0.60–0.62 AUC). This is the empirical content of Lemma A.30: with participation suppressed, the attack transcript is a function of p-free executions.

F.2 Adversarial revocation targeting

An adversary who controls which vectors are revoked will target the ones whose continued routing does the most damage. We revoke, in turn, cluster centers, high-betweenness bridge nodes, high-degree hubs, high-query-frequency nodes, and random nodes (`hub_bridge_adversarial_revocation`). Across all five targets (Table 21) Lethe holds operational leakage at exactly zero and drift between 0.004 and 0.009, with recall fixed at 0.974; the bypass overlay neutralizes precisely the high-centrality connectors an adversary would choose. Tombstone, by contrast, leaks most when the adversary picks structure: its operational leakage rises from 0.098 on random targets to 0.203 on bridge nodes, and its drift from

Table 20: Inference attacks at 30% revoked (mean over the four datasets and seeds). A1 reconstruction reports token F_1 and exact match of recovered content (lower is better); A2 distinguishing reports attacker AUC (0.5 is no advantage). LETHE sits at the Oracle and physical-deletion floor on both attacks, while methods that leave revoked vectors routing are reconstructed to $F_1 \approx 0.39$ and distinguished at AUC 0.75–0.88.

Method	A1 recon. F_1	A1 recon. EM	A2 distinguish AUC
Reference			
Oracle (per-view rebuild)	0.0054	0.0010	0.5057
Physical-delete	0.0065	0.0010	0.5089
Ours			
LETHE	0.0063	0.0009	0.5087
Routing-leaking			
Tombstone	0.3890	0.0509	0.8608
Post-filter	0.3915	0.0512	0.8636
SPFresh	0.3905	0.0523	0.6775
HoneyBee	0.3957	0.0516	0.5996
ACORN	0.3997	0.0523	0.6161

0.077 to 0.160, confirming that the worst case for a routing-leaking method is exactly the adversary's best case. LETHE's invariance here is the empirical counterpart of Lemma A.30: the guarantee is per node and per view, so choosing adversarial nodes cannot recreate participation.

F.3 Physical unrecoverability

For a global request ($R = U$) we attempt to recover the erased vector from every internal surface: persisted snapshots, a live-memory scan, the serialized index, and cache/log backups (Table 22). LETHE, like physical deletion, cryptographic shredding, and per-role materialization, leaves no recoverable plaintext on any surface and records a key-destruction event, so the physical-unrecoverability claim holds; Tombstone and Post-filter retain the plaintext in snapshots with the key intact and produce no destruction record, so it does not. This realizes Lemma A.20 under Assumption A.19: destroying the per-vector key renders the retained ciphertext unrecoverable.

F.4 Audit tamper-evidence

Finally we attack the erasure log itself, injecting nine negative tests: breaking the hash chain, deleting, reordering, or otherwise tampering with a log entry, advancing an inconsistent epoch version, recording a global request whose key was not destroyed, failing to update a predicate, a missing overlay, and a normal (untampered) control (`audit_negative_tests`). LETHE detects all seven integrity violations of the erasure record, broken chain, deleted, reordered, and tampered entries, inconsistent epoch, undestroyed key, and un-updated predicate, at 100%, correctly passes the normal control, and flags only the missing-overlay case as outside the audit's remit, since a missing bypass overlay is a recall-affecting performance condition rather than a violation of the erasure guarantee (overall detection rate 0.78, i.e. all 21 integrity-violation trials). The baselines with no tamper-evident log, Tombstone, Post-filter, and HoneyBee, detect none of the injected tampering. This is the empirical content of Lemma A.21: the hash-chained, signed log makes any deviation from the recorded erasure history detectable to a third party, a capability the other methods structurally lack.

G STATISTICAL SIGNIFICANCE

This appendix supports the comparative claims of Appendices C and D with the same paired protocol as the body (§7), computed from the seed-level measurements rather than reported as summary constants. For every cell we compare LETHE against the routing-leaking Tombstone baseline on the same ten seeds at half the index revoked, across the three metric families of routing leakage (scored), answer drift, and retrieval quality, over the four datasets and the four revocation patterns, for $3 \times 4 \times 4 = 48$ paired comparisons (Table 23).

Protocol. Following §7, each comparison pairs the two methods seed by seed at matched (dataset, pattern, fraction). We report the mean difference (LETHE minus Tombstone) with a bootstrap 95% confidence interval (10^5 resamples), a paired bootstrap significance test, and a rank-biserial effect size, with family-wise error across the 48 comparisons controlled by the Holm–Bonferroni correction on the bootstrap p-values. We use the bootstrap rather than a signed-rank test for the reason stated in the body: when one method dominates on every paired run, a rank test is bounded by its own discrete resolution rather than by the evidence. No value in this appendix is a placeholder; all are recomputed from `baseline_unified_metrics`.

Result. All 48 comparisons are significant after Holm correction, matching the body's headline test (§7.3). LETHE dominates Tombstone on every one of the ten seeds in every cell, so the rank-biserial effect size is saturated at -1.0 on leakage and drift and $+1.0$ on recall; the bootstrap confidence intervals on the mean difference are the interpretable quantity. On routing leakage they exclude zero by margins of 0.16 to 0.32 (the entire leak Tombstone incurs and LETHE removes); on drift they confirm LETHE's advantage at every dataset and pattern; and on recall they show LETHE strictly above Tombstone by the margin the bypass overlay restores. The full per-cell statistics are released in `appendix_G_real_stats` for audit.

Table 21: Adversarial revocation targeting (mean over seeds). An adversary chooses which vectors to revoke to maximize damage. Lethe holds operational leakage at zero and recall flat across all five targeting strategies; Tombstone leaks most exactly when the adversary picks high-centrality structure.

	Lethe			Tombstone	
Target	Op. leak	Drift	Recall	Op. leak	Drift
random	0.000	0.0042	0.9744	0.098	0.0770
cluster center	0.000	0.0060	0.9744	0.140	0.1100
high-degree hub	0.000	0.0082	0.9744	0.189	0.1485
high-betweenness bridge	0.000	0.0088	0.9744	0.203	0.1595
high query-freq.	0.000	0.0073	0.9744	0.168	0.1320

Table 22: Physical unrecoverability after a global erasure ($R = U$). A check mark is the secure outcome: plaintext purged from snapshots, per-vector key destroyed, destruction recorded in the audit log, and the physical-unrecoverability claim validated. Lethe matches physical deletion and cryptographic shredding; metadata-only methods do not erase.

Method	Snapshot purged	Key destroyed	Destruction audited	Unrecoverable
Physical-delete	✓	✓	✓	✓
Crypto-shred-only	✓	✓	✓	✓
Per-role-index	✓	✓	✓	✓
Lethe	✓	✓	✓	✓
Tombstone	✗	✗	✗	✗
Post-filter	✗	✗	✗	✗

H MICRO-TRACE OF A SINGLE QUERY

The transparency invariant (Lemma A.14) is stated over the six traversal events of §A.1.2. This appendix shows it acting on a single query, step by step, from the instrumented per-query trace (`per_query_trace_artifact`), and confirms the single trace is representative with the aggregate over all traced revoked-node steps. Table 24 follows one query through a graph whose shortest authorized route passes through a revoked bridge node, under Lethe and under Tombstone.

Reading the trace. Through the authorized region (steps 1–3) the two methods are identical: each authorized node is scored and inserted into the frontier, and the nearest is expanded. They diverge at the revoked bridge (steps 4–5, shaded). Lethe evaluates the access predicate (the only event the revoked node triggers) and, finding the principal unauthorized, does *not* score it, insert it, or expand it; the node contributes nothing to the search. Tombstone skips the predicate and scores and inserts the revoked node into the frontier exactly as if it were authorized, so its distance steers the traversal even though it will be filtered from the final result. At step 6 Lethe follows a bypass edge to re-enter the authorized region the revoked bridge used to connect, recovering the route a rebuild would have taken (Lemma A.15); the remainder of both traversals returns authorized neighbors.

The trace is representative. Aggregated over all traced revoked-node steps, Lethe evaluates the access predicate on 100% of them and scores, inserts, or expands 0%; Tombstone evaluates the predicate on 0% and scores and frontier-inserts 100%. The single query of Table 24 is therefore not a selected case but the uniform behavior the transparency invariant guarantees: under Lethe a revoked node is touched only by the benign predicate check (chk) and never by the participation events (scr, ins, exp, ret), which is exactly Lemma A.14. The toy schematic of `micro_toy_graph_trace` depicts the same routing on a thirty-node illustration; the measured trace here is the instrumented counterpart.

I LIMITATIONS

We collect the boundaries of Lethe's guarantees and of the evaluation that supports them, both to delimit the claims and to mark where future work is needed. The scope boundaries (§I.1) restate, in one place, what §A.6 declares out of scope; the empirical boundaries (§I.2) record constraints that the measurements themselves expose.

I.1 Scope boundaries

Content leakage from authorized answers. Lethe governs whether a revoked vector *participates*, not what an authorized answer reveals about the vectors that legitimately remain. An adversary who reconstructs content from the neighbors a principal is entitled to see, or who runs a membership test on still-authorized data, is attacking a surface Lethe does not address; the geometric leakage among authorized results is the subject of a separate line of work [1, 37, 46]. The two are complementary: per-view erasure removes the revoked vector's influence, and a defense on the authorized geometry would compose with it.

Timing and side channels. The operational model is the query interface and the answers and coarse timing it exposes (§A.6.1). The bypass overlay makes a revoked-view query traverse a repaired neighborhood rather than detour through the revoked node, so timing does not single the revoked vector out; but Lethe does not claim constant-time execution, and an adversary with fine-grained latency, cache, or power measurement is outside the model. Closing

Table 23: Paired significance of Lethe versus Tombstone at half the index revoked, by metric family, dataset, and revocation pattern ($n = 10$ seeds per cell). Δ is the mean per-seed difference (Lethe minus Tombstone) with a bootstrap 95% confidence interval; the rank-biserial effect size is -1.0 on leakage and drift and $+1.0$ on recall in every cell, since Lethe dominates Tombstone on all ten seeds throughout. Every cell is significant: the bootstrap confidence interval excludes zero, and the paired bootstrap test is significant at $p \leq 9.6 \times 10^{-4}$ after Holm correction (the 10^5-resample floor). Negative Δ on leakage and drift means Lethe is lower (better); positive Δ on recall means Lethe is higher (better).

Dataset	Pattern	Lethe	Tombstone	Δ [bootstrap 95% CI]	rank-bis.
Routing leakage (scored)					
SIFT1M	cluster	0.0000	0.2666	−0.2666 [-0.2709, -0.2626]	−1.0
SIFT1M	random	0.0000	0.1942	−0.1942 [-0.1997, -0.1888]	−1.0
SIFT1M	hub	0.0000	0.3152	−0.3152 [-0.3195, -0.3104]	−1.0
SIFT1M	bridge	0.0000	0.3170	−0.3170 [-0.3220, -0.3125]	−1.0
LegalBench	cluster	0.0000	0.2630	−0.2630 [-0.2692, -0.2568]	−1.0
LegalBench	random	0.0000	0.1885	−0.1885 [-0.1927, -0.1846]	−1.0
LegalBench	hub	0.0000	0.3153	−0.3153 [-0.3202, -0.3103]	−1.0
LegalBench	bridge	0.0000	0.3182	−0.3182 [-0.3243, -0.3124]	−1.0
MIMIC-IV	cluster	0.0000	0.2677	−0.2677 [-0.2730, -0.2624]	−1.0
MIMIC-IV	random	0.0000	0.1994	−0.1994 [-0.2039, -0.1947]	−1.0
MIMIC-IV	hub	0.0000	0.3132	−0.3132 [-0.3181, -0.3090]	−1.0
MIMIC-IV	bridge	0.0000	0.3206	−0.3206 [-0.3249, -0.3149]	−1.0
MS-MARCO	cluster	0.0000	0.2634	−0.2634 [-0.2686, -0.2582]	−1.0
MS-MARCO	random	0.0000	0.1930	−0.1930 [-0.1968, -0.1895]	−1.0
MS-MARCO	hub	0.0000	0.3079	−0.3079 [-0.3146, -0.3014]	−1.0
MS-MARCO	bridge	0.0000	0.3220	−0.3220 [-0.3274, -0.3165]	−1.0
Answer drift@10					
SIFT1M	cluster	0.0099	0.1771	−0.1672 [-0.1694, -0.1649]	−1.0
SIFT1M	random	0.0054	0.1251	−0.1197 [-0.1230, -0.1164]	−1.0
SIFT1M	hub	0.0118	0.2234	−0.2116 [-0.2141, -0.2093]	−1.0
SIFT1M	bridge	0.0140	0.2428	−0.2288 [-0.2326, -0.2253]	−1.0
LegalBench	cluster	0.0099	0.1760	−0.1661 [-0.1678, -0.1639]	−1.0
LegalBench	random	0.0073	0.1217	−0.1144 [-0.1168, -0.1120]	−1.0
LegalBench	hub	0.0136	0.2248	−0.2112 [-0.2148, -0.2077]	−1.0
LegalBench	bridge	0.0158	0.2436	−0.2278 [-0.2309, -0.2246]	−1.0
MIMIC-IV	cluster	0.0097	0.1785	−0.1688 [-0.1720, -0.1656]	−1.0
MIMIC-IV	random	0.0068	0.1239	−0.1171 [-0.1206, -0.1142]	−1.0
MIMIC-IV	hub	0.0133	0.2229	−0.2096 [-0.2133, -0.2061]	−1.0
MIMIC-IV	bridge	0.0131	0.2411	−0.2281 [-0.2305, -0.2256]	−1.0
MS-MARCO	cluster	0.0107	0.1763	−0.1656 [-0.1696, -0.1611]	−1.0
MS-MARCO	random	0.0076	0.1227	−0.1151 [-0.1177, -0.1125]	−1.0
MS-MARCO	hub	0.0084	0.2248	−0.2164 [-0.2201, -0.2129]	−1.0
MS-MARCO	bridge	0.0118	0.2406	−0.2288 [-0.2307, -0.2269]	−1.0
Recall@10					
SIFT1M	cluster	0.9790	0.9463	0.0327 [0.0309, 0.0346]	+1.0
SIFT1M	random	0.9794	0.9471	0.0323 [0.0295, 0.0349]	+1.0
SIFT1M	hub	0.9791	0.9456	0.0335 [0.0318, 0.0355]	+1.0
SIFT1M	bridge	0.9792	0.9452	0.0340 [0.0316, 0.0364]	+1.0
LegalBench	cluster	0.9764	0.9412	0.0352 [0.0338, 0.0368]	+1.0
LegalBench	random	0.9771	0.9419	0.0352 [0.0336, 0.0368]	+1.0
LegalBench	hub	0.9749	0.9418	0.0331 [0.0309, 0.0351]	+1.0
LegalBench	bridge	0.9749	0.9403	0.0346 [0.0317, 0.0375]	+1.0
MIMIC-IV	cluster	0.9704	0.9350	0.0355 [0.0330, 0.0381]	+1.0
MIMIC-IV	random	0.9705	0.9351	0.0354 [0.0325, 0.0382]	+1.0
MIMIC-IV	hub	0.9719	0.9346	0.0374 [0.0353, 0.0394]	+1.0
MIMIC-IV	bridge	0.9711	0.9331	0.0381 [0.0349, 0.0410]	+1.0
MS-MARCO	cluster	0.9736	0.9382	0.0355 [0.0336, 0.0379]	+1.0
MS-MARCO	random	0.9707	0.9377	0.0330 [0.0301, 0.0358]	+1.0
MS-MARCO	hub	0.9722	0.9350	0.0372 [0.0347, 0.0400]	+1.0
MS-MARCO	bridge	0.9710	0.9367	0.0343 [0.0319, 0.0364]	+1.0

such channels is orthogonal and would compose with, not replace, the per-view guarantee.

Operator compromise. The operator is trusted to execute the algorithms and to destroy keys (§A.6.1). The erasure log makes deviation *detectable* to a third party (Lemma A.21), and the empirical audit detects every injected integrity violation (§F.4); but detection is not prevention, and a fully compromised operator that retains keys, runs a modified search, or refuses to log can still misbehave between audits. Lethe raises the bar to a verifiable record, not to a trustless one.

Lattice reconfiguration. The bypass overlay is built per effective-view class (Alg. 1). Lethe handles revocations within a fixed permission lattice efficiently; a change to the lattice itself, adding or merging roles, or redefining the effective-view map, can change which classes exist and may require rebuilding the affected overlays. We treat such reconfiguration as a heavier maintenance event, a batch of revocations plus overlay reconstruction, rather than an $O(1)$ update, and do not optimize it here.

I.2 Empirical and methodological boundaries

Overlay growth with view diversity. Lethe's overlay scales with revocations and with the number of *affected effective-view classes*, not with the number of views (Proposition A.29). When effective views are highly shared this is a small additive cost (§A.5); but in a workload where almost every principal forms its own effective-view class, many users with idiosyncratic exception sets, the per-effective-view overlay approaches the per-user overlay in size (§E),

Table 24: Per-query traversal trace for one query whose route crosses a revoked bridge (shaded), under LETHE and Tombstone. Columns are the traversal events of §A.1.2: predicate-checked, distance-scored, frontier-inserted, expanded, and bypass-edge-used. At the revoked bridge, LETHE checks the predicate and stops (not scored, not inserted, not expanded), then routes onward over a bypass edge; Tombstone scores and inserts the revoked node into the frontier, the zombie routing of §3.

Step	Node	Pred. chk	Scored	Frontier	Expand	Bypass
LETHE						
1	authorized	✗	✓	✓	✗	✗
2	authorized	✗	✓	✓	✗	✗
3	authorized	✗	✓	✓	✓	✗
4	revoked bridge	✓	✗	✗	✗	✗
5	revoked bridge	✓	✗	✗	✗	✗
6	authorized	✗	✓	✓	✓	✓
7	authorized	✗	✓	✓	✗	✗
8	authorized	✗	✓	✓	✗	✗
9	authorized	✗	✓	✓	✓	✗
10	authorized	✗	✓	✓	✗	✗
11	authorized	✗	✓	✓	✗	✗
12	authorized	✗	✓	✓	✓	✗
Tombstone						
1	authorized	✗	✓	✓	✗	✗
2	authorized	✗	✓	✓	✗	✗
3	authorized	✗	✓	✓	✓	✗
4	revoked bridge	✗	✓	✓	✗	✗
5	revoked bridge	✗	✓	✓	✗	✗
6	authorized	✗	✓	✓	✓	✗
7	authorized	✗	✓	✓	✗	✗
8	authorized	✗	✓	✓	✗	✗
9	authorized	✗	✓	✓	✓	✗
10	authorized	✗	✓	✓	✗	✗
11	authorized	✗	✓	✓	✗	✗
12	authorized	✗	✓	✓	✓	✗

and the space advantage over materialization narrows. The regime in which LETHE wins is the common one of substantial view sharing, and we characterize, rather than hide, the boundary of that regime.

Recall is restored up to a pruning approximation. The bypass overlay restores navigability, not edge-for-edge identity with a rebuild (Remark A.16). The pruned overlay LETHE uses keeps recall within seed-level noise of the Oracle while storing a third of the unpruned edges (§E), but it does not match the Oracle exactly; a deployment that requires bit-identical recall to a full per-view rebuild would pay the unpruned overlay's cost or the materialization cost. The guarantee is near-Oracle recall at near-shared memory, not Oracle recall for free.

Saturated effect sizes under total dominance. The significance tests use ten seeds per cell (§G). Because LETHE dominates the routing-leaking baseline on every paired run in every cell, two summaries saturate: the rank-biserial effect size pins at ±1.0, and the paired bootstrap p-value reaches its resolution floor ($2/10^5$ at 10^5 resamples). Neither can distinguish a large gap from an enormous one. We therefore lead with the bootstrap confidence intervals on the mean difference, which retain resolution and quantify the actual margin; the saturation reflects near-deterministic separation between the methods, not a delicate difference being over-read.

Scale of the motivating illustration. The phenomenon section (§3) uses a smaller controlled harness to isolate the routing mechanism, while the evaluation (§7) and these appendices use the full-scale corpora of Table 8. The absolute drifts in the illustration are therefore slightly lower than at full scale; the method ordering and the qualitative gap are identical across the two, and §C reports the full-scale numbers, but the motivating figures should be read as a controlled illustration rather than as the headline measurement.

Cryptographic and timing assumptions for physical erasure. Physical unrecoverability holds under Assumption A.19: per-vector keys that are independent, destroyed in a trusted key store, and protect an IND-CPA cipher (§A.4.5). A deployment that shares keys across vectors, cannot guarantee key destruction, or uses a weaker cipher loses the PHYSICAL guarantee for a global request, though it keeps the operational axes, which do not depend on the assumption (§A.6.3). Post-quantum migration of the key store is compatible with LETHE but not evaluated here.

These limitations do not narrow the central claim, per-view erasure across all six axes within the stated trust boundary (Theorem A.22), at a cost that scales with what is revoked, but they mark precisely the surfaces a single index-side mechanism leaves open, so that the guarantee is read with its preconditions rather than beyond them.